Practical Number: 1

LED Blinking Patterns and Telegram-Based GPIO Control Using Raspberry Pi 5

Program:

```
from telegram import Update
from telegram.ext import Application, CommandHandler, ContextTypes
from gpiozero import LED
from time import sleep
BOT_TOKEN = ""
# LED configuration
led1 = LED(17)
led2 = LED(27)
led3 = LED(22)
led4 = LED(23)
# Turn all LEDs OFF
def all_off():
    led1.off()
    led2.off()
    led3.off()
    led4.off()
# Pattern 1: Sequential blinking
def pattern1():
    for led in [led1, led2, led3, led4]:
        led.on()
        sleep(0.5)
        led.off()


# Pattern 2: All LEDs blink together
def pattern2():
    led1.on()
    led2.on()
    led3.on()
    led4.on()
    sleep(1)
    all_off()
# Pattern 3: Alternate blinking
def pattern3():
    # LED 1 and LED 3 ON
    led1.on()
    led3.on()
    led2.off()
```

```
    led4.off()
    sleep(1)
    # LED 2 and LED 4 ON
    led1.off()
    led3.off()
    led2.on()
    led4.on()
    sleep(1)
    all_off()
# Pattern 4: Running light
def pattern4():
    for led in [led1, led2, led3, led4]:
        led.on()
        sleep(0.3)
        led.off()
# /start command
async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    message = (
        "Raspberry Pi 5 LED Controller\n\n"
        "/pattern1 - Sequential blinking\n"
        "/pattern2 - All LEDs blink\n"
        "/pattern3 - Alternate blinking\n"
        "/pattern4 - Running light\n"
        "/off - Turn all LEDs OFF"
    )
    await update.message.reply_text(message)
# /pattern1 command
async def pattern1_command(
    update: Update,
    context: ContextTypes.DEFAULT_TYPE
):
    pattern1()
    await update.message.reply_text(
        "Pattern 1 executed: Sequential blinking"
    )
# /pattern2 command
async def pattern2_command(
    update: Update,
    context: ContextTypes.DEFAULT_TYPE
):
    pattern2()
```

```
    await update.message.reply_text(
        "Pattern 2 executed: All LEDs blinking"
    )
# /pattern3 command
async def pattern3_command(
    update: Update,
    context: ContextTypes.DEFAULT_TYPE
):
    pattern3()
    await update.message.reply_text(
        "Pattern 3 executed: Alternate blinking"
    )
# /pattern4 command
async def pattern4_command(
    update: Update,
    context: ContextTypes.DEFAULT_TYPE
):
    pattern4()
    await update.message.reply_text(
        "Pattern 4 executed: Running light"
    )
# /off command
async def off_command(
    update: Update,
    context: ContextTypes.DEFAULT_TYPE
):
    all_off()
    await update.message.reply_text(
        "All LEDs turned OFF"
    )
# Create Telegram application
app = Application.builder().token(BOT_TOKEN).build()
# Register commands
app.add_handler(CommandHandler("start", start))
app.add_handler(CommandHandler("pattern1", pattern1_command))
app.add_handler(CommandHandler("pattern2", pattern2_command))
app.add_handler(CommandHandler("pattern3", pattern3_command))
app.add_handler(CommandHandler("pattern4", pattern4_command))
app.add_handler(CommandHandler("off", off_command))
print("Telegram 4-LED Controller Running...")
# Start bot
```

app.run_polling()

Output:
Raspberry Pi 5 LED Controller

/pattern1 - Sequential blinking

/pattern2 - All LEDs blink

/pattern3 - Alternate blinking

/pattern4 - Running light

/off - Turn all LEDs OFF

PROTEUSE:

Program

import RPi.GPIO as GPIO

from time import sleep

GPIO.setmode(GPIO.BCM)

LED1 = 4

LED2 = 7

LED3 = 5

LED4 = 6

```
GPIO.setup(LED1, GPIO.OUT)
GPIO.setup(LED2, GPIO.OUT)
GPIO.setup(LED3, GPIO.OUT)
GPIO.setup(LED4, GPIO.OUT)
while True:
    # LEDs ON
    GPIO.output(LED1, GPIO.HIGH)
    GPIO.output(LED2, GPIO.HIGH)
    GPIO.output(LED3, GPIO.HIGH)
    GPIO.output(LED4, GPIO.HIGH)
    sleep(1)
    # LEDs OFF
    GPIO.output(LED1, GPIO.LOW)
    GPIO.output(LED2, GPIO.LOW)
    GPIO.output(LED3, GPIO.LOW)
    GPIO.output(LED4, GPIO.LOW)
    sleep(1)
```

Output:

Fig 1.1

Circuit Connection

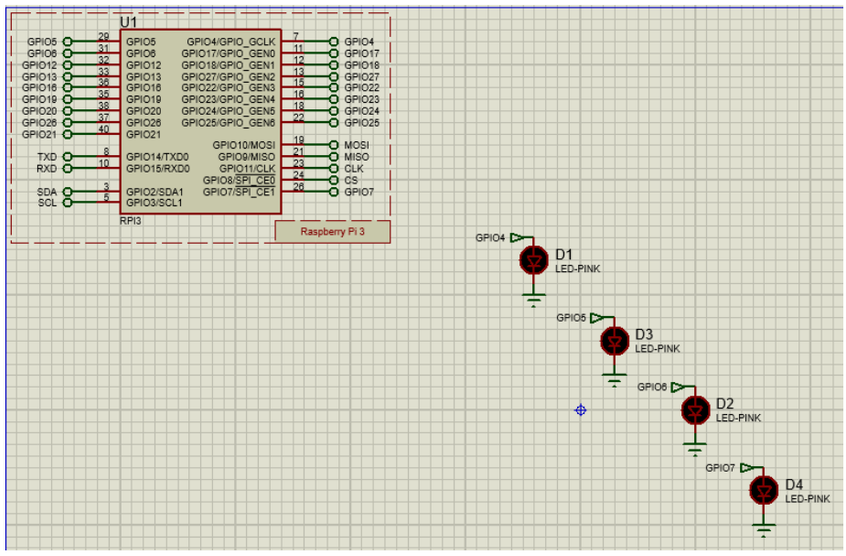
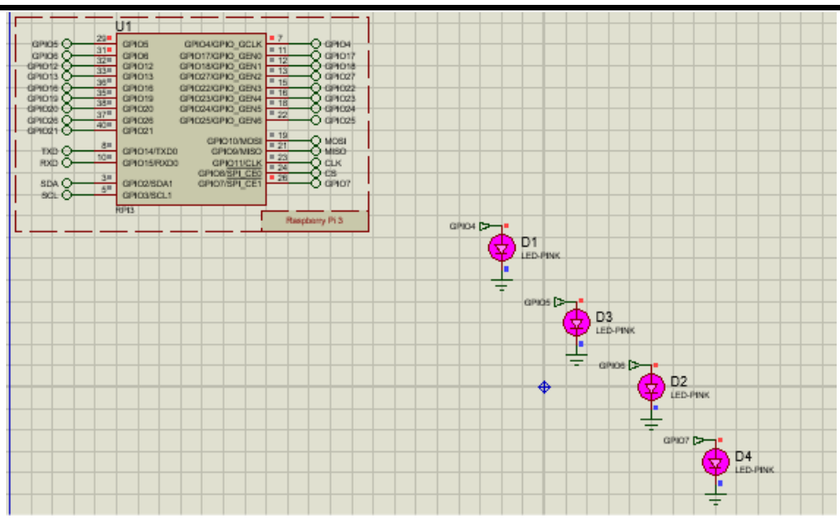


Fig 1.2

Output for blinking LED's



Practical Number: 2

Use Telegram to control Raspberry Pi GPIO.

Program:

```
from telegram import Update
from telegram.ext import Application, CommandHandler, ContextTypes
from gpiozero import LED
from time import sleep
BOT_TOKEN = ""
# LED configuration
led1 = LED(17)
led2 = LED(27)
led3 = LED(22)
led4 = LED(23)
# Turn all LEDs OFF
def all_off():
    led1.off()
    led2.off()
    led3.off()
    led4.off()
# Pattern 1: Sequential blinking
def pattern1():
```

```
    for led in [led1, led2, led3, led4]:
        led.on()
        sleep(0.5)
        led.off()
# Pattern 2: All LEDs blink together
def pattern2():
    led1.on()
    led2.on()
    led3.on()
    led4.on()
    sleep(1)
    all_off()
# Pattern 3: Alternate blinking
def pattern3():
    # LED 1 and LED 3 ON
    led1.on()
    led3.on()
    led2.off()
    led4.off()
    sleep(1)
    # LED 2 and LED 4 ON
    led1.off()
    led3.off()
    led2.on()
    led4.on()
    sleep(1)
    all_off()
# Pattern 4: Running light
def pattern4():
    for led in [led1, led2, led3, led4]:
        led.on()
        sleep(0.3)
        led.off()
# /start command
async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    message = (
        "Raspberry Pi 5 LED Controller\n\n"
        "/pattern1 - Sequential blinking\n"
        "/pattern2 - All LEDs blink\n"
        "/pattern3 - Alternate blinking\n"
        "/pattern4 - Running light\n"
```

```
        "/off - Turn all LEDs OFF"

    await update.message.reply_text(message)
# /pattern1 command
async def pattern1_command(
    update: Update,
    context: ContextTypes.DEFAULT_TYPE
):
    pattern1()
    await update.message.reply_text(
        "Pattern 1 executed: Sequential blinking"
    )
# /pattern2 command
async def pattern2_command(
    update: Update,
    context: ContextTypes.DEFAULT_TYPE
):
    pattern2()
    await update.message.reply_text(
        "Pattern 2 executed: All LEDs blinking"
    )
# /pattern3 command
async def pattern3_command(
    update: Update,
    context: ContextTypes.DEFAULT_TYPE
):
    pattern3()
    await update.message.reply_text(
        "Pattern 3 executed: Alternate blinking"
    )
# /pattern4 command
async def pattern4_command(
    update: Update,
    context: ContextTypes.DEFAULT_TYPE
):
    pattern4()
    await update.message.reply_text(
        "Pattern 4 executed: Running light"
    )
# /off command
async def off_command(
```

```
    update: Update,
    context: ContextTypes.DEFAULT_TYPE
):
    all_off()
    await update.message.reply_text(
        "All LEDs turned OFF"
    )
# Create Telegram application
app = Application.builder().token(BOT_TOKEN).build()
# Register commands
app.add_handler(CommandHandler("start", start))
app.add_handler(CommandHandler("pattern1", pattern1_command))
app.add_handler(CommandHandler("pattern2", pattern2_command))
app.add_handler(CommandHandler("pattern3", pattern3_command))
app.add_handler(CommandHandler("pattern4", pattern4_command))
app.add_handler(CommandHandler("off", off_command))
print("Telegram 4-LED Controller Running...")
# Start bot
app.run_polling()
```

Output:

```
Raspberry Pi 5 LED Controller
/pattern1 - Sequential blinking
/pattern2 - All LEDs blink
/pattern3 - Alternate blinking
/pattern4 - Running light
/off - Turn all LEDs OFF
```

Practical Number:3

Interface TM1637 with Raspberry Pi and Display current time.

Code:

```
from tm1637 import TM1637
from datetime import datetime
from time import sleep

# Initialize the TM1637 display (CLK on GPIO 23, DIO on GPIO 24)
display = TM1637(clk=23, dio=24)

while True:
    # Get current time in HHMM format for display
    current_time = datetime.now().strftime("%H%M")

    # Show time on the 4-digit 7-segment display
    display.show(current_time)

    # Print time to console in HH:MM:SS format
    print("Current Time:", datetime.now().strftime("%H:%M:%S"))

    # Wait for 1 second before updating
    sleep(1)
```

Output:

10:35

Code: Display Date

```
from tm1637 import TM1637
from datetime import datetime
display = TM1637(clk=23, dio=24)
date = datetime.now().strftime("%d%m")
display.show(date)
print("Date:", datetime.now().strftime("%d-%m-%Y"))
```

Output:

```
TM1637:
27 08
```

Code: Display Temperature

```
from tm1637 import TM1637
from gpiozero import CPUTemperature
from time import sleep
display = TM1637(clk=23, dio=24)
cpu = CPUTemperature()

while True:
    temperature = int(cpu.temperature)

    display.number(temperature)

    print("CPU Temperature:", temperature, "°C")

    sleep(1)
```

Output:

```
 49
```

Code: Display Hexadecimal Value

```
from tm1637 import TM1637

display = TM1637(clk=23, dio=24)

number = 255
hex_value = hex(number)[2:].upper()

display.show(hex_value)
```

```
print("Hexadecimal:", hex_value)
```

Output:

TM1637:

00FF

Code: Display Binary Value

```
from tm1637 import TM1637
display = TM1637(clk=23, dio=24)
number = 10
binary_value = bin(number)[2:]
binary_value = binary_value.zfill(4)
display.show(binary_value)
print("Binary:", binary_value)
```

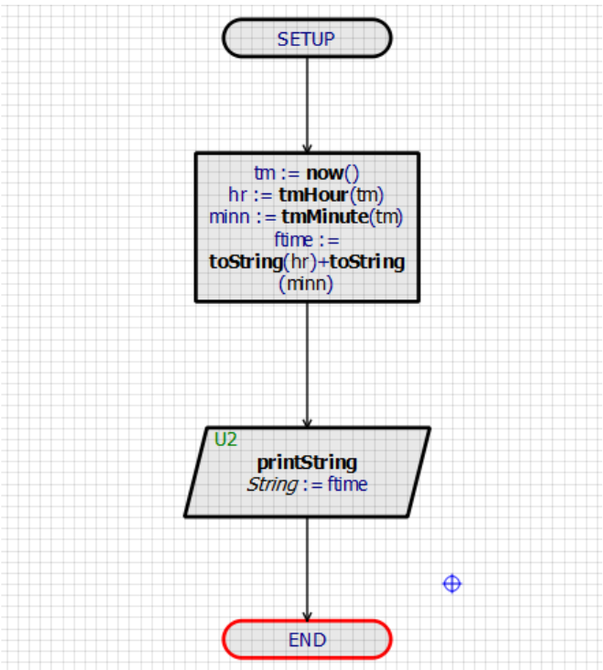
Output:

Binary:

1010

Flowchart:

Fig 3.1



Output:

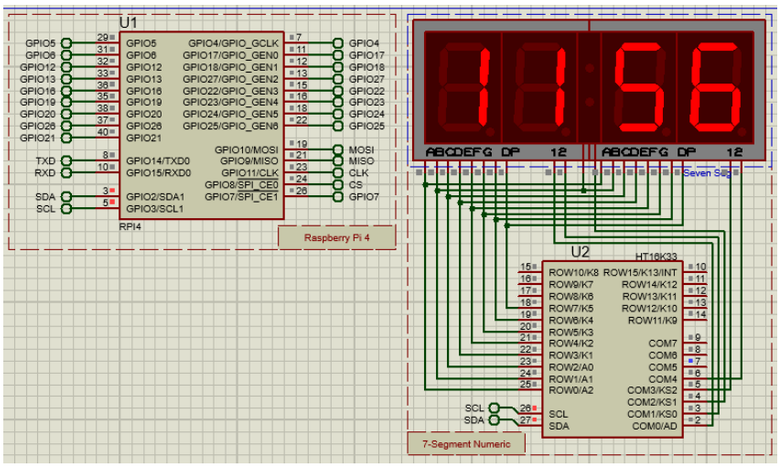
Fig 3.2

Output Showing the Current Time



Fig 3.3

Circuit Connection



Practical Number:6

Interface 16x2 LCD with Raspberry Pi using I2C module to display text.

Program

```
from RPLCD.i2c import CharLCD
import psutil
from gpiozero import CPUTemperature
from time import sleep
lcd = CharLCD('PCF8574', 0x27)
cpu_temp = CPUTemperature()
while True:
    # Read system information
    cpu = psutil.cpu_percent(interval=None)
    memory = psutil.virtual_memory().percent
    temperature = cpu_temp.temperature
    # Screen 1: CPU and Memory
    lcd.clear()
    lcd.write_string(f"CPU:{cpu:.1f}%")
    lcd.cursor_pos = (1, 0)
    lcd.write_string(f"MEM:{memory:.1f}%")
    sleep(2)
    # Screen 2: Temperature
    lcd.clear()
```

```
    lcd.write_string(f"TEMP:{temperature:.1f} C")
    lcd.cursor_pos = (1, 0)
    lcd.write_string("Raspberry Pi 5")
    sleep(2)
```

Output:

Screen 1

CPU:18.5%

MEM:42.3%

and:

Screen 2

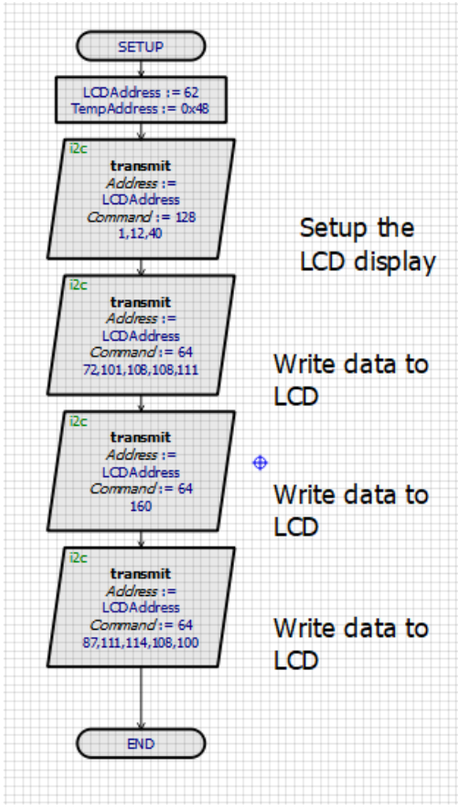
TEMP:49.8 C

Raspberry Pi 5

PROTEUSE:

Flowchart:

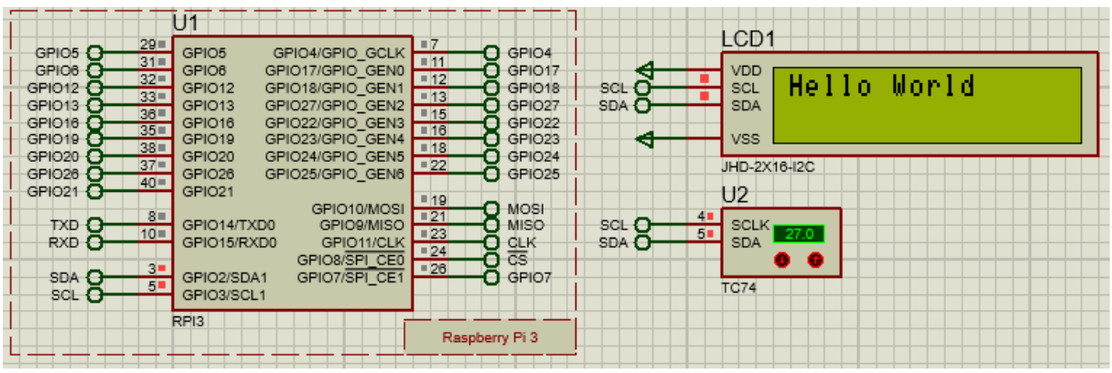
Fig 6.1



Output:

Fig 6.2

Output Displaying Text



Practical Number:18

Write a program to read the CPU%, Memory% and CPU temperature of Raspberry Pi.

Program

```
from RPLCD.i2c import CharLCD
import psutil
from gpiozero import CPUTemperature
from time import sleep
lcd = CharLCD('PCF8574', 0x27)
cpu_temp = CPUTemperature()
while True:
    # Read system information
    cpu = psutil.cpu_percent(interval=None)
    memory = psutil.virtual_memory().percent
    temperature = cpu_temp.temperature
    # Screen 1: CPU and Memory
    lcd.clear()
    lcd.write_string(f"CPU:{cpu:.1f}%")
    lcd.cursor_pos = (1, 0)
    lcd.write_string(f"MEM:{memory:.1f}%")
    sleep(2)
    # Screen 2: Temperature
    lcd.clear()
    lcd.write_string(f"TEMP:{temperature:.1f} C")
    lcd.cursor_pos = (1, 0)
    lcd.write_string("Raspberry Pi 5")
    sleep(2)
```

Output:

Screen 1

CPU:18.5%

MEM:42.3%

and:

Screen 2

TEMP:49.8 C

Raspberry Pi 5

Practical Number: 9

Interface Pi Camera with Raspberry Pi to capture image and video.

Flowchart: For Clicking An Image

Fig 9.1

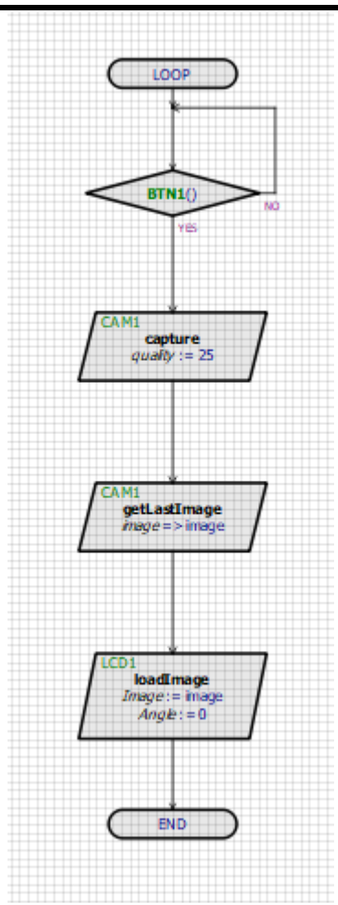


Fig 9.2

Output showing Image

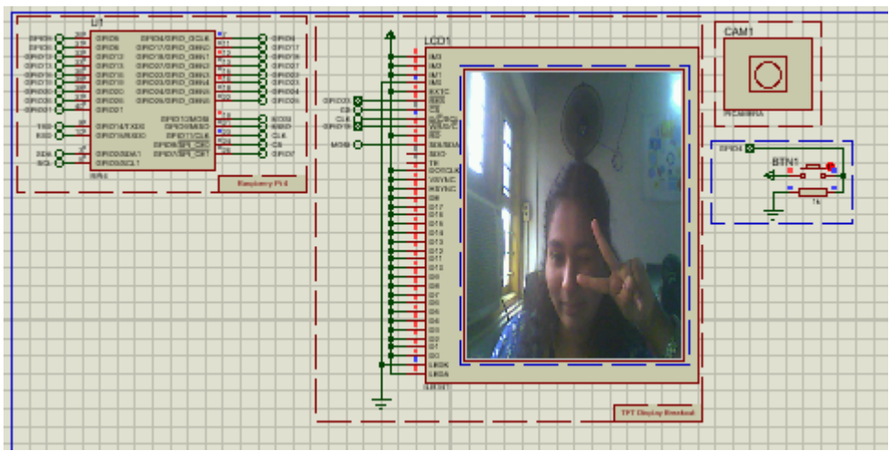


Fig 9.3

Output showing Image on TFT Display



Flowchart: For Taking a Video

Fig 9.4

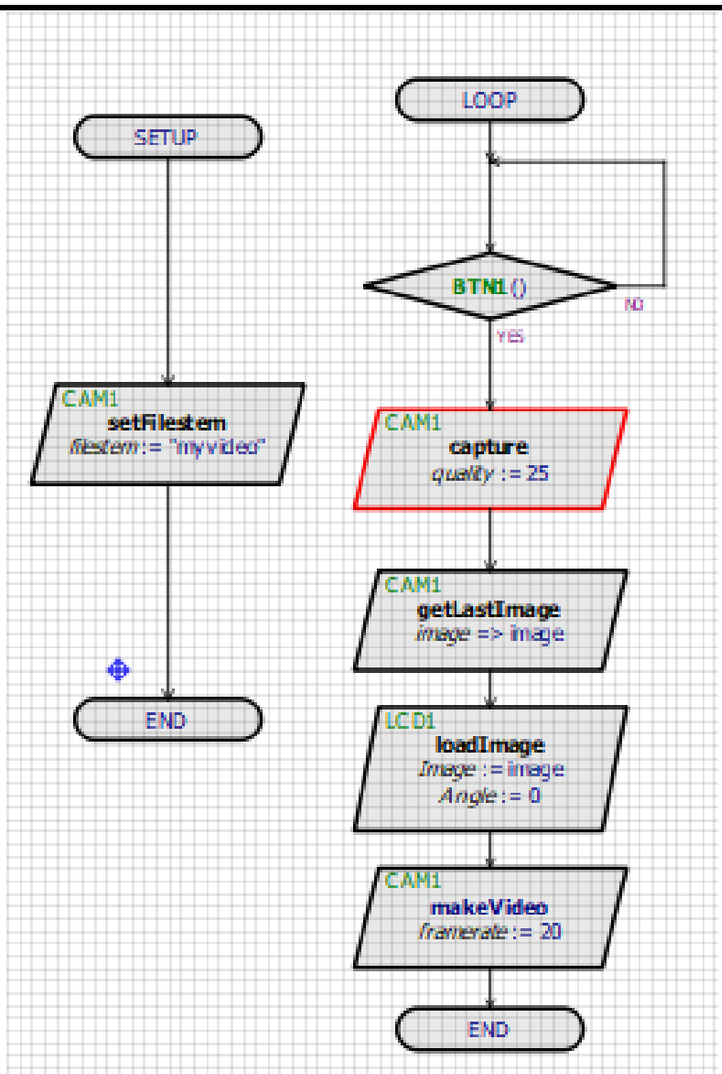


Fig 9.5

Output



Practical Number: 9

Interface Pi Camera with Raspberry Pi to capture image and video.

Code 1: Capture Image

```
from picamera2 import Picamera2
```

```
import time
picam2 = Picamera2()
config = picam2.create_still_configuration()
picam2.configure(config)
picam2.start()
time.sleep(2)
picam2.capture_file("image.jpg")
picam2.stop()
print("Image Captured Successfully")
```

Output

Image Captured Successfully

Code 2: Record Video

```
from picamera2 import Picamera2
from picamera2.encoders import H264Encoder
import time
picam2 = Picamera2()
video_config = picam2.create_video_configuration()
picam2.configure(video_config)
encoder = H264Encoder()
picam2.start()
picam2.start_recording(
    encoder,
    "video.h264"
)
time.sleep(10)

picam2.stop_recording()
picam2.stop()
print("Video Recorded Successfully")
```

Run

Output

Video Recorded Successfully

Practical Number: 7

Interface RFID module with Raspberry Pi to read and write to RFID tags/cards.

Flowchart:

Fig 7.1

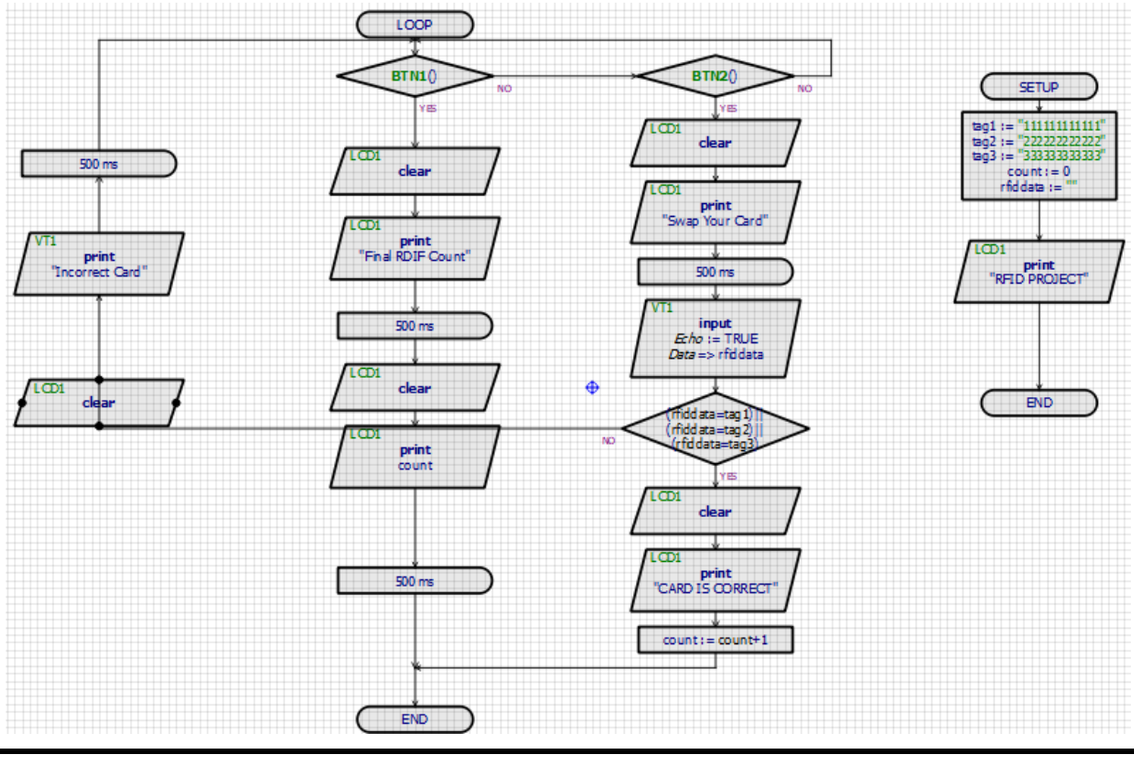


Fig 7.2

RFID Circuit Connection

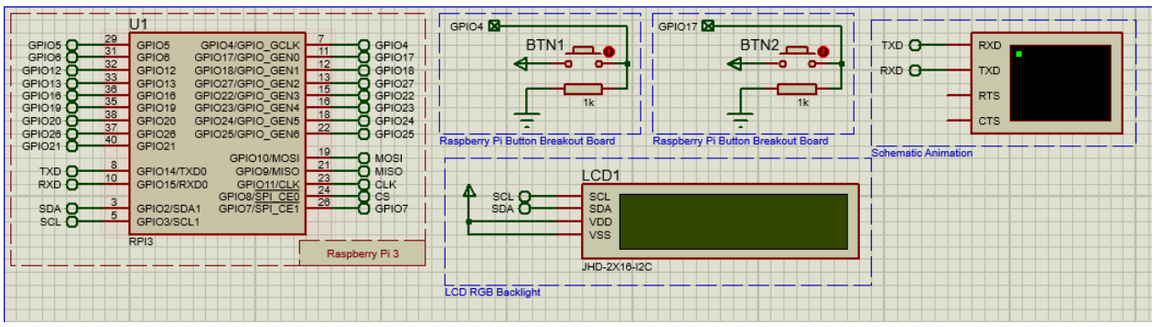


Fig 7.3

First Message

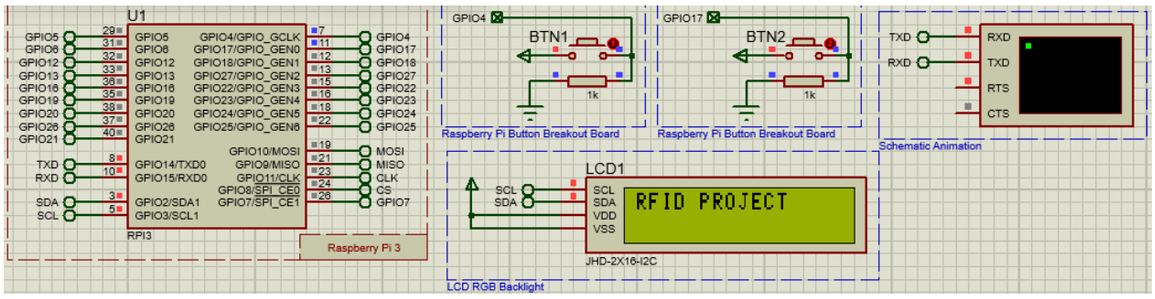


Fig 7.4

Asking for your Card details

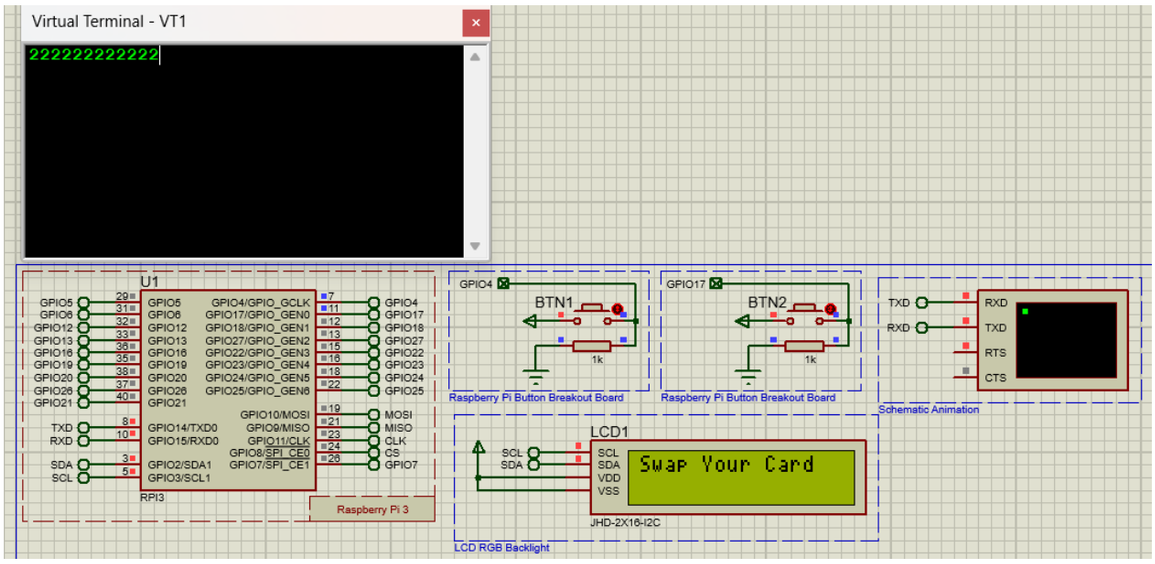


Fig 7.5

Message if your card is correct

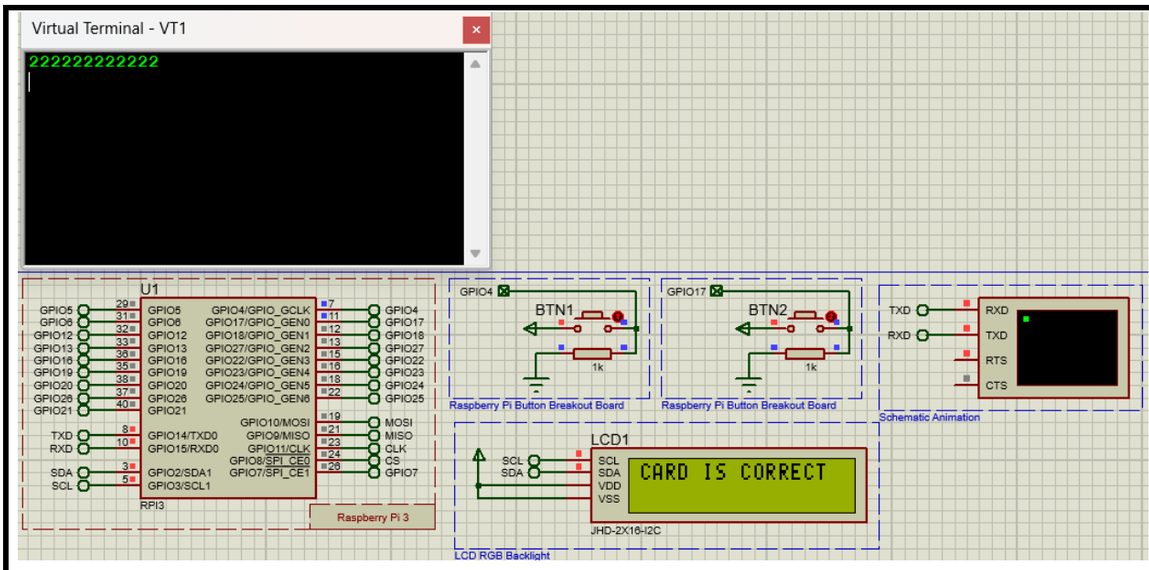


Fig 7.6

Message if Card is Wrong

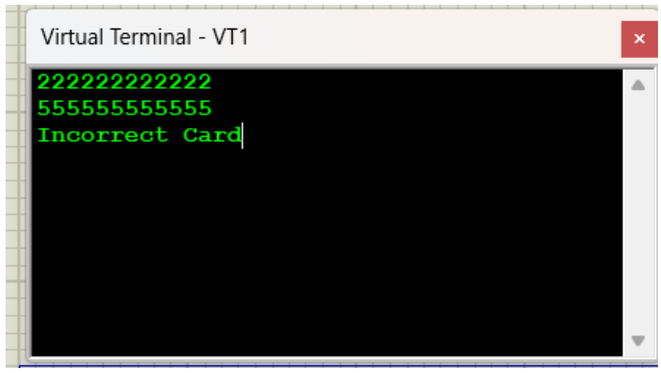
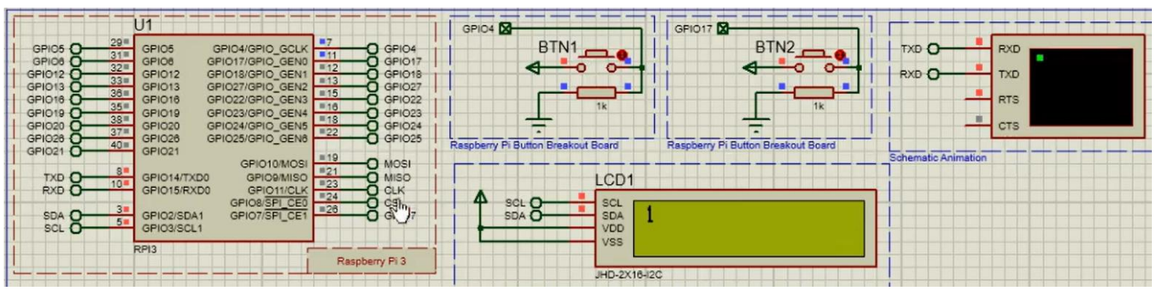


Fig 7.7

Showing RFID Count



Practical Number: 7

Interface RFID module with Raspberry Pi to read and write to RFID tags/cards.

Code 1: Read RFID UID

```
from JoyIT_RC522 import SimpleMFRC522
reader = SimpleMFRC522()
print("Place RFID card near the reader...")
card_id = reader.read_id()
print("Card UID:", card_id)
```

Output:

Place RFID card near the reader...

Card UID: 123456789

Code 2: Write Data to RFID Card

```
from JoyIT_RC522 import SimpleMFRC522
reader = SimpleMFRC522()
text = input("Enter text: ")
print("Place RFID card near the reader...")
reader.write(text)
print("Data written successfully.")
```

Output:

Enter text: Raspberry Pi RFID

Place RFID card near the reader...

Data written successfully.

Code 3: Read Stored RFID Data

```
from JoyIT_RC522 import SimpleMFRC522
```

```
reader = SimpleMFRC522()
print("Place RFID card near the reader...")
card_id, text = reader.read()
print("Card UID:", card_id)
print("Stored Data:", text)
```

Output:

Place RFID card near the reader...

Card UID: 123456789

Stored Data: Raspberry Pi RFID

Practical Number: 4

Interface GPS module with Raspberry Pi.

PROTUSE:

Code:

```
import time
import pio
import Ports
pio.uart = Ports.UART()
pio.uart.setup(9600)
while True:
    Data = pio.uart.recv()
    if Data:
        pio.uart.print(Data)
```

Output:

Fig 4.1

Figure showing Circuit

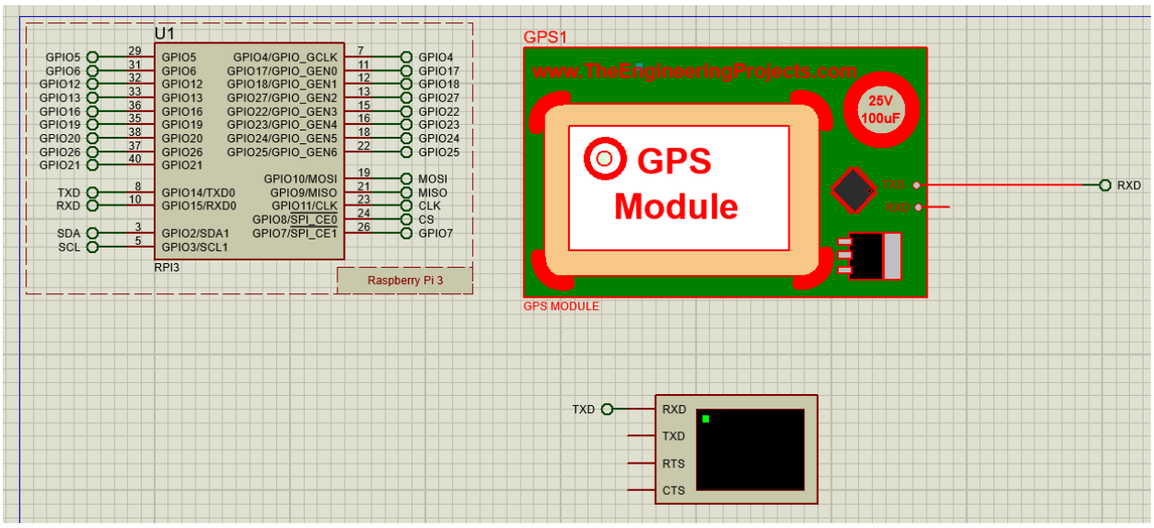
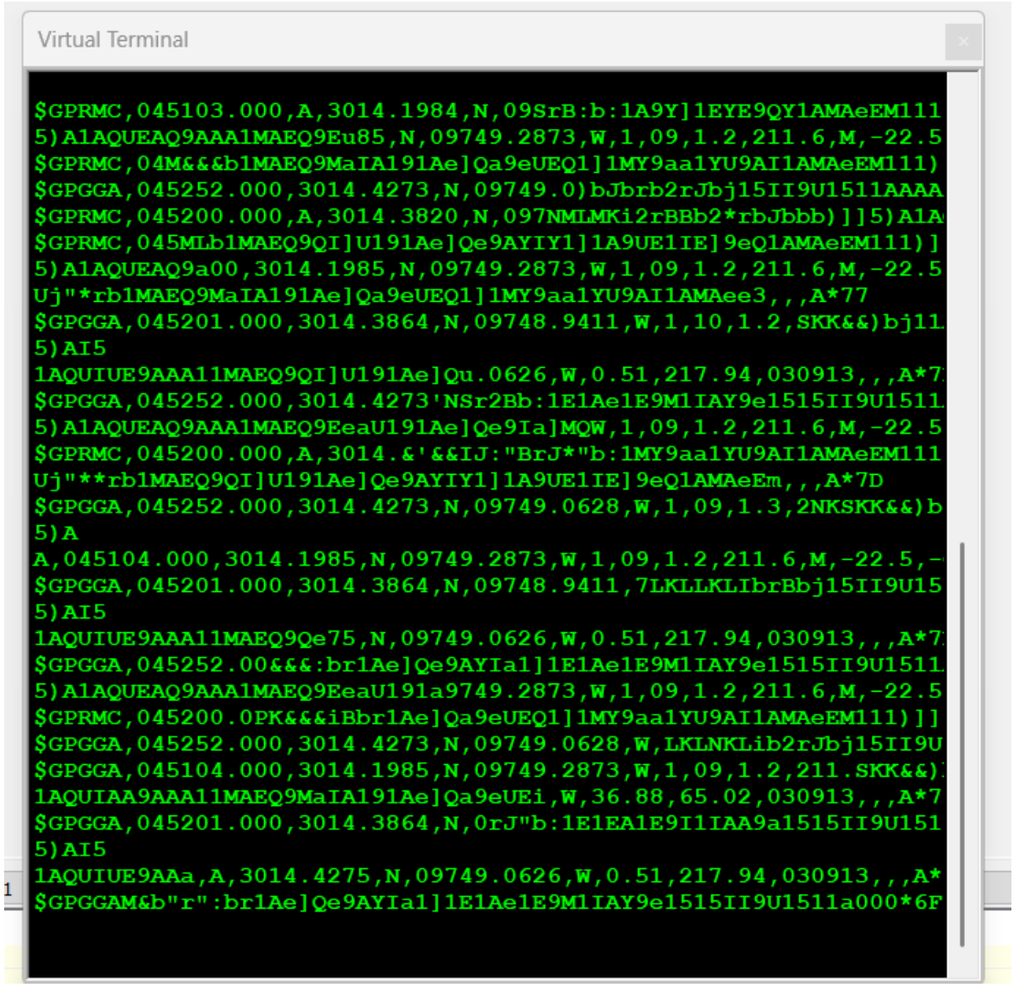


Fig 4.2

Output showing Coordinates



Practical Number: 4

Interface GPS module with Raspberry Pi.

To get Raw GPS Data Configure UART:

sudo stty -F /dev/ttyAMA0 9600 cs8 -cstopb -parenb

Read GPS data

sudo timeout 30 cat /dev/ttyAMA0

Code 1: Read Raw GPS Data Using Python

```
import serial
gps = serial.Serial(
    port="/dev/ttyAMA0",
    baudrate=9600,
    timeout=1
)
print("GPS serial connection started...")
print("Reading NMEA data...\n")
try:
    while True:
        line = gps.readline().decode(
            "ascii",
            errors="replace"
        ).strip()
        if line:
            print(line)
except KeyboardInterrupt:
    print("\nGPS program stopped.")
finally:
    gps.close()
```

Code2. GPS Data With Latitude, Longitude, Time, Speed

```
import serial
import pynmea2
import time

gps = serial.Serial(
    port="/dev/ttyAMA0",
    baudrate=9600,
    timeout=1
)
print("GPS data reader started...")
```

```
print("Waiting for GPS fix...\n")
last_status = 0
try:
    while True:
        line = gps.readline().decode(
            "ascii",
            errors="replace"
        ).strip()
        if not line:
            continue
        try:
            message = pynmea2.parse(line)
            if isinstance(
                message,
                pynmea2.types.talker.RMC
            ):
                if message.status == "A":
                    print("\n----- GPS FIX -----")
                    print("Latitude :", message.latitude)
                    print("Longitude:", message.longitude)
                    print("Time     :", message.timestamp)
                    print("Speed    :", message.spd_over_grnd)
                    print("Course   :", message.true_course)
                else:
                    if time.time() - last_status >= 5:
                        print(
                            "Waiting for valid GPS fix..."
                        )
                        last_status = time.time()
        except pynmea2.ParseError:
            pass
except KeyboardInterrupt:
    print("\nGPS program stopped.")
finally:
    gps.close()
```

Practical Number:5

Interface ADS 1115 with Raspberry Pi to act as oscilloscope.

Code 1: Read A0

```
import time
import board
from adafruit_ads1x15 import ADS1115
from adafruit_ads1x15 import AnalogIn
from adafruit_ads1x15 import ads1x15
i2c = board.I2C()
ads = ADS1115(i2c)
channel = AnalogIn(
    ads,
    ads1x15.Pin.A0
)
print("ADS1115 detected!")
print("Reading A0...")
while True:
    print(
        "Raw:", channel.value,
        "Voltage:", round(channel.voltage, 3), "V"
    )
    time.sleep(0.5)
```

Code 2: Read A0–A3

```
import time
import board
from adafruit_ads1x15 import ADS1115
from adafruit_ads1x15 import AnalogIn
from adafruit_ads1x15 import ads1x15
i2c = board.I2C()
ads = ADS1115(i2c)
a0 = AnalogIn(
    ads,
    ads1x15.Pin.A0
)
a1 = AnalogIn(
    ads,
    ads1x15.Pin.A1
)
a2 = AnalogIn(
    ads,
    ads1x15.Pin.A2
)
a3 = AnalogIn(
    ads,
    ads1x15.Pin.A3
)
print("ADS1115 Multi-Channel Reading Started")
while True:
    print("\n-------------------------")
    print(
        "A0:",
        round(a0.voltage, 3),
        "V"
    )
    print(
        "A1:",
        round(a1.voltage, 3),
        "V"
    print(
        "A2:",
        round(a2.voltage, 3),
        "V"
    )
```

```
    print(
        "A3:",
        round(a3.voltage, 3),
        "V"
    )
    time.sleep(1)
```

Code 3: Live Voltage Graph

```
import time
import board
import matplotlib.pyplot as plt
from adafruit_ads1x15 import ADS1115
from adafruit_ads1x15 import AnalogIn
from adafruit_ads1x15 import ads1x15
i2c = board.I2C()
ads = ADS1115(i2c)
channel = AnalogIn(
    ads,
    ads1x15.Pin.A0
)
times = []
voltages = []
start_time = time.time()
plt.ion()
fig, ax = plt.subplots()
line, = ax.plot([], [])
ax.set_xlabel("Time (seconds)")
ax.set_ylabel("Voltage (V)")
ax.set_title("ADS1115 - A0 Voltage")
print("ADS1115 Graph Started")
print("Close the graph window or press Ctrl+C to stop.")
try:
    while plt.fignum_exists(fig.number):
        voltage = channel.voltage
        current_time = time.time() - start_time
        times.append(current_time)
        voltages.append(voltage)
        line.set_xdata(times)
        line.set_ydata(voltages)
        ax.relim()
        ax.autoscale_view()
        plt.draw()
```

```
        plt.pause(0.5)
except KeyboardInterrupt:
    print("\nProgram stopped by user.")
finally:
    plt.close("all")
    print("ADS1115 program stopped.")
```

Practical Number:8

Interface Fingerprint Sensor with Raspberry Pi.

Code 1: Test R307 Connection

```
from pyfingerprint.pyfingerprint import PyFingerprint
try:
    sensor = PyFingerprint(
        '/dev/serial0',
        57600,
        0xFFFFFFFF,
        0x00000000
    )
    if sensor.verifyPassword():
        print(
            "Fingerprint sensor "
            "connected successfully!"
        )
    else:
        print(
            "Fingerprint sensor "
            "password verification failed."
        )
except Exception as e:
    print(
        "Fingerprint sensor "
        "could not be initialized!"
    )
    print("Error:", e)
```

Output:

Fingerprint sensor connected successfully!

Fingerprint Code 2: Capture Fingerprint

```
from pyfingerprint.pyfingerprint import PyFingerprint
import time
try:
    sensor = PyFingerprint(
        '/dev/serial0',
        57600,
        0xFFFFFFFF,
        0x0000000   )
    if sensor.verifyPassword():
        print("Fingerprint sensor connected successfully!   “     )
    else:
        print("Password verification failed.")
        exit()
    print("Place your finger on the sensor..."   )
    while not sensor.readImage():
        time.sleep(0.1)
    print("Fingerprint captured successfully!"  )
except Exception as e:
    print("Error:", e)
```

Output:

```
Fingerprint sensor connected successfully!
Place your finger on the sensor...
First fingerprint captured!
Remove your finger...
Place the SAME finger again...
Second fingerprint captured!
Fingerprints matched!
Fingerprint enrolled successfully!
Stored at position: 0
```

Practical No:12

Simulate door lock by interfacing fingerprint sensor and relay moule with raspberry Pi.
Code:

Enroll.py

```
import serial
import adafruit_fingerprint


uart = serial.Serial(
    "/dev/serial0",
    baudrate=57600,
    timeout=1
)


finger = adafruit_fingerprint.Adafruit_Fingerprint(uart)


def get_fingerprint_image():
    print("Place finger on sensor...")

    while True:
        result = finger.get_image()

        if result == adafruit_fingerprint.OK:
            print("Image captured.")
            return True

        elif result == adafruit_fingerprint.NOFINGER:
            continue

        else:
            print("Error:", result)
            return False


try:
    location = int(input("Enter fingerprint ID (1-27): "))

    if location < 1 or location > 27:
        print("ID must be between 1 and 27.")
        raise SystemExit

    print("\n--- First Scan ---")
```

```
if not get_fingerprint_image():
    raise SystemExit

result = finger.image_2_tz(1)

if result != adafruit_fingerprint.OK:
    print("Could not convert first image.")
    raise SystemExit

print("Remove your finger...")

while finger.get_image() != adafruit_fingerprint.NOFINGER:
    pass

print("\n--- Second Scan ---")
print("Place the SAME finger again...")

if not get_fingerprint_image():
    raise SystemExit

result = finger.image_2_tz(2)

if result != adafruit_fingerprint.OK:
    print("Could not convert second image.")
    raise SystemExit

print("Creating fingerprint model...")

result = finger.create_model()

if result != adafruit_fingerprint.OK:
    print("Fingerprints did not match.")
    raise SystemExit

print("Storing fingerprint...")

result = finger.store_model(location)

if result == adafruit_fingerprint.OK:
    print("\nFingerprint enrolled successfully!")
    print("Fingerprint ID:", location)
```

```
    else:
        print("Could not store fingerprint.")
        print("Error:", result)

finally:
    uart.close()
    print("Serial connection closed.")
```

Output:

```
(fingerprint-env) iot2026@raspberrypi:~/fingerprint-door-lock $ python Enroll.py

Enter fingerprint ID (1-27): 1

--- First Scan ---
Place finger on sensor...
Image captured.
Remove your finger...

--- Second Scan ---
Place the SAME finger again...
Place finger on sensor...
Image captured.

Creating fingerprint model...
Storing fingerprint...

Fingerprint enrolled successfully!
Fingerprint ID: 1
Serial connection closed.
```

door_lock.py

```
import time
import serial
import adafruit_fingerprint
from gpiozero import OutputDevice

UART_PORT = "/dev/serial0"
BAUDRATE = 57600

RELAY_PIN = 17
UNLOCK_TIME = 5

relay = OutputDevice(
    RELAY_PIN,
    active_high=False,
    initial_value=False
)

uart = serial.Serial(
    UART_PORT,
    baudrate=BAUDRATE,
    timeout=1
)

finger = adafruit_fingerprint.Adafruit_Fingerprint(uart)

def check_fingerprint():

    print("\nPlace your finger on the sensor...")

    while True:
        result = finger.get_image()

        if result == adafruit_fingerprint.OK:
            print("Image captured.")
            break

        elif result == adafruit_fingerprint.NOFINGER:
            time.sleep(0.1)

        elif result == adafruit_fingerprint.IMAGEFAIL:
```

```
            print("Fingerprint image error.")
            return False

        else:
            print("Sensor error:", result)
            return False

    result = finger.image_2_tz(1)

    if result != adafruit_fingerprint.OK:
        print("Could not convert fingerprint image.")
        return False

    result = finger.finger_search()

    if result == adafruit_fingerprint.OK:
        print("        ACCESS GRANTED")

        print("Fingerprint ID:", finger.finger_id)
        print("Confidence:", finger.confidence)

        return True

    elif result == adafruit_fingerprint.NOTFOUND:
        print("        ACCESS DENIED")

        print("Fingerprint not recognized.")
        return False

    else:
        print("Fingerprint search error:", result)
        return False

def unlock_door():

    print("\nUnlocking door...")

    relay.on()

    print("Relay ON")
    print("Door unlocked for", UNLOCK_TIME, "seconds.")
```

```
    time.sleep(UNLOCK_TIME)

    relay.off()

    print("Relay OFF")
    print("Door locked.")

try:

    print(" Raspberry Pi Fingerprint Door Lock")

    print("\nChecking fingerprint sensor...")

    result = finger.read_sysparam()

    if result == adafruit_fingerprint.OK:
        print("Fingerprint sensor connected.")
        print("Stored fingerprints:", finger.template_count)

    else:
        print("Could not communicate with fingerprint sensor.")
        raise SystemExit

    while True:
        authorized = check_fingerprint()

        if authorized:
            unlock_door()
        else:
            print("Door remains locked.")
            time.sleep(1)

except KeyboardInterrupt:
    print("\nProgram stopped by user.")

finally:
    relay.off()
    uart.close()

    print("Relay OFF")
```

```
    print("Program terminated.")
```

Output:

```
(fingerprint-env) iot2026@raspberrypi:~/fingerprint-door-lock $ python door_lock.py
 Raspberry Pi Fingerprint Door Lock

Checking fingerprint sensor...
Fingerprint sensor connected.
Stored fingerprints: 1

Place your finger on the sensor...
Image captured.

       ACCESS GRANTED

Fingerprint ID: 1
Confidence: 80

Unlocking door...
Relay ON
Door unlocked for 5 seconds.
Relay OFF
Door locked.
```

Practical No:14

Design a web dashboard using Flask. Control relay module via web.

app.py

```
from flask import Flask, render_template, jsonify
from gpiozero import OutputDevice
import threading
import time

app = Flask(__name__)

RELAY_PIN = 17
UNLOCK_TIME = 5

relay = OutputDevice(
    RELAY_PIN,
    active_high=False,
    initial_value=False
)

def lock_door():
    relay.off()
    print("Door LOCKED")

def unlock_door():
    relay.on()
    print("Door UNLOCKED")
    print("Door will automatically lock after", UNLOCK_TIME, "seconds")

    threading.Timer(
        UNLOCK_TIME,
        lock_door
    ).start()

@app.route("/")
def index():
    status = "UNLOCKED" if relay.is_active else "LOCKED"

    return render_template(
        "index.html",
        status=status
    )
```

```
@app.route("/unlock", methods=["POST"])
def unlock():
    unlock_door()

    return jsonify({
        "success": True,
        "status": "UNLOCKED",
        "message": "Door unlocked for 5 seconds"
    })

@app.route("/lock", methods=["POST"])
def lock():
    lock_door()

    return jsonify({
        "success": True,
        "status": "LOCKED",
        "message": "Door locked"
    })

@app.route("/status")
def status():
    return jsonify({
        "status": "UNLOCKED" if relay.is_active else "LOCKED"
    })

if __name__ == "__main__":

    try:
        print("====================================")
        print(" Raspberry Pi Door Lock Server")
        print("====================================")
        print("Relay GPIO:", RELAY_PIN)
        print("Unlock time:", UNLOCK_TIME, "seconds")
        print("Server starting...")

        app.run(
            host="0.0.0.0",
            port=5080,
            debug=False
```

```
    )

except KeyboardInterrupt:
    print("\nServer stopped.")

finally:
    relay.off()

    print("Relay OFF")
    print("Door LOCKED")
```

templates/index.html

```
<!DOCTYPE html>
<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport"
content="width=device-width, initial-scale=1.0">

<title>Raspberry Pi Smart Door</title>

<style>

* {
    box-sizing: border-box;
}

body {
    margin: 0;
    min-height: 100vh;
    font-family: Arial, sans-serif;
    background: linear-gradient(135deg, #0f172a, #10293b);
    color: white;
    display: flex;
    justify-content: center;
    align-items: center;
}

.container {
    width: 90%;
    max-width: 600px;
}

.header {
    text-align: center;
    margin-bottom: 25px;
}

.header h1 {
```

```
    margin-bottom: 8px;
    font-size: 32px;
}

.header p {
    color: #94a3b8;
}

.card {
    background: rgba(255, 255, 255, 0.08);
    border: 1px solid rgba(255, 255, 255, 0.1);
    border-radius: 20px;
    padding: 30px;
    text-align: center;
    box-shadow: 0 25px 50px rgba(0, 0, 0, 0.3);
    backdrop-filter: blur(10px);
}

.lock-icon {
    font-size: 70px;
    margin-bottom: 15px;
}

.status {
    font-size: 28px;
    font-weight: bold;
    margin-bottom: 25px;
}

.locked {
    color: #22c55e;
}

.unlocked {
    color: #f59e0b;
}

.buttons {
    display: flex;
    justify-content: center;
    gap: 15px;
```

```
}

button {
    border: none;
    padding: 14px 25px;
    border-radius: 10px;
    font-size: 16px;
    font-weight: bold;
    cursor: pointer;
    transition: 0.2s;
}

button:hover {
    transform: scale(1.05);
}

.unlock {
    background: #22c55e;
    color: white;
}

.lock {
    background: #ef4444;
    color: white;
}

.message {
    margin-top: 25px;
    min-height: 25px;
    color: #cbd5e1;
}

.info {
    margin-top: 25px;
    padding: 15px;
    background: rgba(0, 0, 0, 0.2);
    border-radius: 18px;
    text-align: left;
    color: #cbd5e1;
}
```

```
@media (max-width: 500px) {

    .buttons {
        flex-direction: column;
    }

    button {
        width: 100%;
    }
}

</style>

</head>

<body>

<div class="container">

<div class="header">

<h1>Smart Door Lock</h1>

<p>Raspberry Pi Security Dashboard</p>

</div>

<div class="card">

<div id="lockIcon" class="lock-icon">
🔒
</div>

<div id="status" class="status locked">
LOCKED
</div>

<div class="buttons">

<button
class="unlock"
```

```
onclick="unlockDoor()">

🔓 Unlock Door

</button>

<button
class="lock"
onclick="lockDoor()">

🔒 Lock Door

</button>

</div>

<div id="message" class="message">

System ready

</div>

<div class="info">

<b>System</b><br>
Raspberry Pi GPIO 17<br>
Relay-controlled door lock<br>
Automatic locking after 5 seconds

</div>

</div>

</div>

<script>

function updateUI(status) {

    const statusElement =
        document.getElementById("status");
```

```
    const icon =
        document.getElementById("lockIcon");

    if (status === "UNLOCKED") {

        statusElement.innerText = "UNLOCKED";

        statusElement.className =
            "status unlocked";

        icon.innerText = " 🔓 ";

    } else {

        statusElement.innerText = "LOCKED";

        statusElement.className =
            "status locked";

        icon.innerText = " 🔒 ";
    }
}

function unlockDoor() {

    document.getElementById("message").innerText =
        "Unlocking door...";

    fetch("/unlock", {
        method: "POST"
    })

    .then(response => response.json())

    .then(data => {

        updateUI(data.status);

        document.getElementById("message").innerText =
            data.message;
```

```
  })

  .catch(error => {

    document.getElementById("message").innerText =
      "Connection error";

    console.error(error);

  });
}

function lockDoor() {

  document.getElementById("message").innerText =
    "Locking door...";

  fetch("/lock", {
    method: "POST"
  })

  .then(response => response.json())

  .then(data => {

    updateUI(data.status);

    document.getElementById("message").innerText =
      data.message;

  })

  .catch(error => {

    document.getElementById("message").innerText =
      "Connection error";

    console.error(error);

  });
```

```
}

function getStatus() {

    fetch("/status")

    .then(response => response.json())

    .then(data => {

        updateUI(data.status);

    })

    .catch(error => {

        console.error(error);

    });
}

setInterval(getStatus, 2000);

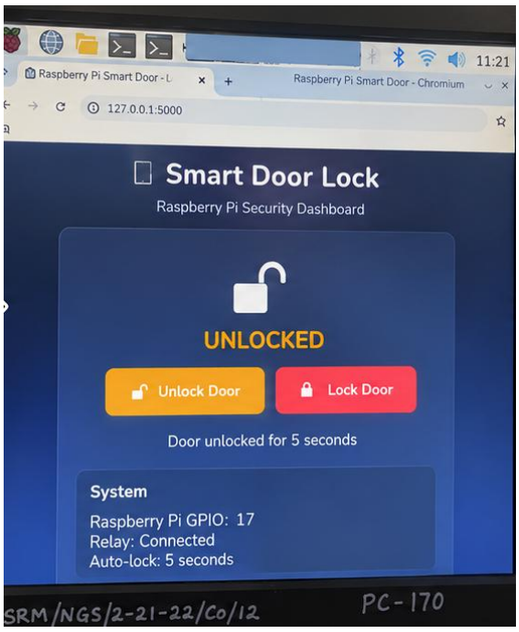
getStatus();

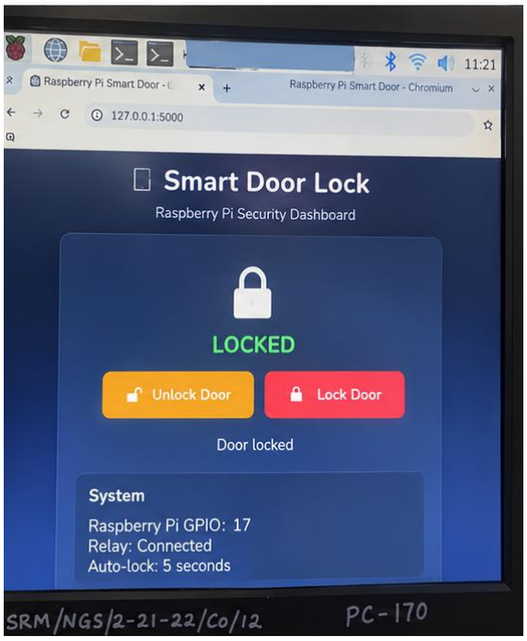
</script>

</body>

</html>
```

Outptut:


11:21
Raspberry Pi Smart Door - L
Raspberry Pi Smart Door - Chromium
127.0.0.1:5000
Smart Door Lock
Raspberry Pi Security Dashboard
UNLOCKED
Unlock Door
Lock Door
Door unlocked for 5 seconds
System
Raspberry Pi GPIO: 17
Relay: Connected
Auto-lock: 5 seconds
SRM/NGS/2-21-22/Co/12
PC-170


11:21
Raspberry Pi Smart Door - L
Raspberry Pi Smart Door - Chromium
127.0.0.1:5000
Smart Door Lock
Raspberry Pi Security Dashboard
LOCKED
Unlock Door
Lock Door
Door locked
System
Raspberry Pi GPIO: 17
Relay: Connected
Auto-lock: 5 seconds
SRM/NGS/2-21-22/Co/12
PC-170

Practical no:11

11. Interface Fingerprint Sensor and 16X2 LCD with Raspberry and display message RFID UID matched or unmatched

Code: lcd16x2.py

```
import smbus
import time


I2C_ADDR = 0x27
LCD_WIDTH = 16


LCD_CHR = 1
LCD_CMD = 0


LCD_LINE_1 = 0x80
LCD_LINE_2 = 0xC0


LCD_BACKLIGHT = 0x08
ENABLE = 0b00000100


BUS = 1


bus = smbus.SMBus(BUS)


def lcd_toggle_enable(bits):
    time.sleep(0.0005)


    bus.write_byte(I2C_ADDR, bits | ENABLE)
    time.sleep(0.0005)


    bus.write_byte(I2C_ADDR, bits & ~ENABLE)
    time.sleep(0.0005)


def lcd_byte(bits, mode):
    high_bits = mode | (bits & 0xF0) | LCD_BACKLIGHT
    low_bits = mode | ((bits << 4) & 0xF0) | LCD_BACKLIGHT


    bus.write_byte(I2C_ADDR, high_bits)
    lcd_toggle_enable(high_bits)


    bus.write_byte(I2C_ADDR, low_bits)
    lcd_toggle_enable(low_bits)
```

```
def lcd_init():
    lcd_byte(0x33, LCD_CMD)
    lcd_byte(0x32, LCD_CMD)
    lcd_byte(0x06, LCD_CMD)
    lcd_byte(0x0C, LCD_CMD)
    lcd_byte(0x28, LCD_CMD)
    lcd_byte(0x01, LCD_CMD)

    time.sleep(0.005)

def lcd_clear():
    lcd_byte(0x01, LCD_CMD)
    time.sleep(0.002)

def lcd_display_string(message, line):
    if line == 1:
        lcd_byte(LCD_LINE_1, LCD_CMD)
    elif line == 2:
        lcd_byte(LCD_LINE_2, LCD_CMD)

    message = message.ljust(LCD_WIDTH)[:LCD_WIDTH]

    for char in message:
        lcd_byte(ord(char), LCD_CHR)

def lcd_message(line1, line2):
    lcd_clear()
    lcd_display_string(line1, 1)
    lcd_display_string(line2, 2)

def lcd_cleanup():
    lcd_clear()
    bus.close()
```

Code:rfid_lcd.py

```
import time
import RPi.GPIO as GPIO
from mfrc522 import SimpleMFRC522

from lcd16x2 import (
    lcd_init,
    lcd_message,
    lcd_clear,
    lcd_cleanup
)

AUTHORIZED_UID = "B9-86-28-03"

reader = SimpleMFRC522()

lcd_init()

print("=" * 50)
print("       RFID + FINGERPRINT + I2C LCD")
print("=" * 50)

print("Initializing I2C LCD...")
print("LCD initialized.")

print("Initializing RFID reader...")
print("RFID reader initialized.")

print()
print("=" * 50)
print("             SYSTEM READY")
print("=" * 50)

print()
print("Step 1: Scan RFID card.")
print("Step 2: Scan fingerprint.")
print()
print("Press Ctrl+C to stop.")

lcd_message("RFID SYSTEM", "READY")
```

```
try:

    while True:

        print()
        print("-" * 50)
        print("             RFID CARD DETECTED")
        print("-" * 50)

        lcd_message("RFID CARD", "DETECTING...")

        id, text = reader.read()

        uid = format(id, 'X').upper()

        uid_bytes = id.to_bytes(4, byteorder='big')

        uid_string = "-".join(
            "{:02X}".format(byte)
            for byte in uid_bytes
        )

        print("RFID UID:", uid_string)

        if uid_string == AUTHORIZED_UID:

            print("RFID UID MATCHED")
            print("RFID authentication successful.")

            lcd_message(
                "RFID UID",
                "MATCHED"
            )

            time.sleep(2)

            lcd_message(
                "ACCESS",
                "AUTHORIZED"
            )
```

```
            time.sleep(2)

        else:

            print("RFID UID UNMATCHED")
            print("RFID authentication failed.")

            lcd_message(
                "RFID UID",
                "UNMATCHED"
            )

            time.sleep(2)

            lcd_message(
                "ACCESS",
                "DENIED"
            )

            time.sleep(2)

        lcd_message(
            "SCAN RFID",
            "CARD..."
        )

        print()
        print("Ready for next authentication.")

        time.sleep(1)

except KeyboardInterrupt:

    print()
    print("Program stopped.")

finally:

    lcd_cleanup()
    GPIO.cleanup()
```

```
    print("GPIO/SPI closed.")
    print("Program completed.")
```

Output:

```
iot2026@raspberrypi:~/rfid-lcd $ source rfid-env/bin/activate

(rfid-env) iot2026@raspberrypi:~/rfid-lcd $ python rfid_lcd.py

==================================================
      RFID + FINGERPRINT + I2C LCD
==================================================
Initializing I2C LCD...
LCD initialized.
Initializing RFID reader...
RFID reader initialized.

==================================================
           SYSTEM READY
==================================================

Step 1: Scan RFID card.
Step 2: Scan fingerprint.

Press Ctrl+C to stop.



--------------------------------------------------
           RFID CARD DETECTED
--------------------------------------------------
RFID UID: B9-86-28-03
RFID UID MATCHED
RFID authentication successful.

Ready for next authentication.



--------------------------------------------------
           RFID CARD DETECTED
--------------------------------------------------
RFID UID: A1-B2-C3-D4
RFID UID UNMATCHED
```

RFID authentication failed.

Ready for next authentication.

RFID CARD DETECTED

RFID UID: B9-86-28-03

RFID UID MATCHED

RFID authentication successful.

Ready for next authentication.

RFID CARD DETECTED

RFID UID: 12-34-56-78

RFID UID UNMATCHED

RFID authentication failed.

Ready for next authentication.

Practical No:16

Create a motion detector by interfacing camera and 16X2 display with Raspberry Pi. Print the appropriate message on the display.

pir_test.py

```
from gpiozero import MotionSensor
from time import sleep

pir = MotionSensor(17)

print("PIR Sensor Ready")

while True:
    if pir.motion_detected:
        print("Motion Detected")
    else:
        print("No Motion")

    sleep(1)
```

motion_detector.py

```
from gpiozero import MotionSensor
from picamera2 import Picamera2
from RPLCD.i2c import CharLCD
import time
import os

PIR_PIN = 17
LCD_ADDRESS = 0x27
IMAGE_FOLDER = "/home/pi/motion_images"

os.makedirs(IMAGE_FOLDER, exist_ok=True)

pir = MotionSensor(PIR_PIN)

lcd = CharLCD(
    i2c_expander='PCF8574',
    address=LCD_ADDRESS,
    port=1,
    cols=16,
    rows=2
)
```

```
def display_message(line1, line2=""):
    lcd.clear()

    lcd.cursor_pos = (0, 0)
    lcd.write_string(line1[:16])

    lcd.cursor_pos = (1, 0)
    lcd.write_string(line2[:16])

camera = Picamera2()

config = camera.create_still_configuration(
    main={"size": (1280, 720)}
)

camera.configure(config)
camera.start()

time.sleep(3)

display_message(
    "MOTION DETECTOR",
    "SYSTEM READY"
)

print("----------------------------")
print("Motion Detector Started")
print("PIR    : GPIO17")
print("LCD    : I2C")
print("Camera : Ready")
print("----------------------------")

time.sleep(2)

display_message(
    "NO MOTION",
    "MONITORING..."
)

try:
```

```
while True:

    pir.wait_for_motion()

    print("Motion detected!")

    display_message(
        "MOTION DETECTED",
        "CAPTURING..."
    )

    timestamp = time.strftime(
        "%Y%m%d_%H%M%S"
    )

    image_path = os.path.join(
        IMAGE_FOLDER,
        "motion_" + timestamp + ".jpg"
    )

    print("Capturing image...")

    camera.capture_file(image_path)

    print("Image saved:")
    print(image_path)

    display_message(
        "MOTION DETECTED",
        "IMAGE SAVED"
    )

    pir.wait_for_no_motion()

    print("No motion.")

    display_message(
        "NO MOTION",
        "MONITORING..."
    )
```

```
        time.sleep(0.5)

except KeyboardInterrupt:

    print("Program stopped.")

finally:

    display_message(
        "SYSTEM STOPPED",
        "GOODBYE!"
    )

    time.sleep(2)

    lcd.clear()
    camera.stop()

    print("Camera stopped.")
    print("LCD cleared.")
```

Output:

```
(motion-env) iot2026@raspberrypi:~/motion-detector $ python motion_detector.py


----------------------------
Motion Detector Started
PIR    : GPIO17
LCD    : I2C
Camera : Ready
----------------------------

Motion detected!
Capturing image...
Image saved:
/home/pi/motion_images/motion_20261007_190512.jpg

No motion.

Motion detected!
Capturing image...
```

Image saved:

/home/pi/motion_images/motion_20261007_190527.jpg

No motion.

Motion detected!

Capturing image...

Image saved:

/home/pi/motion_images/motion_20261007_190545.jpg

No motion.

Practical No:17

Design and implement a Node-RED flow on Raspberry Pi that integrates a physical push-button and an LED with a web dashboard. The flow must allow users to toggle the LED from the dashboard and via the physical button. A button press should force the LED ON for 5 seconds and then automatically restore the LED to the dashboard's last state

Fig 17.1Command – node-red-start

```
user@DESKTOP-4K1D8MT:/pi$ node-red-start

Start Node-RED

Once Node-RED has started, point a browser at http://192.168.1.224:1880
On Pi Node-RED works better with the Firefox or Chrome browser

Use   node-red-stop                          to stop Node-RED
Use   node-red-start                         to start Node-RED again
Use   node-red-log                           to view the recent log output
Use   sudo systemctl enable nodered.service  to autostart Node-RED at every boot
Use   sudo systemctl disable nodered.service to disable autostart on boot

To find more nodes and example flows, go to http://flows.nodered.org

Starting as a systemd service.
4 Oct 06:59:10  - [info] Started flows
4 Oct 07:04:08  - [info] Stopping flows
4 Oct 07:04:08  - [info] Stopped flows
4 Oct 07:04:08  - [info] Updated flows
4 Oct 07:04:08  - [info] Starting flows
4 Oct 07:04:08  - [info] Started flows
```

Fig 17.2: Take 1 inject for Led On message

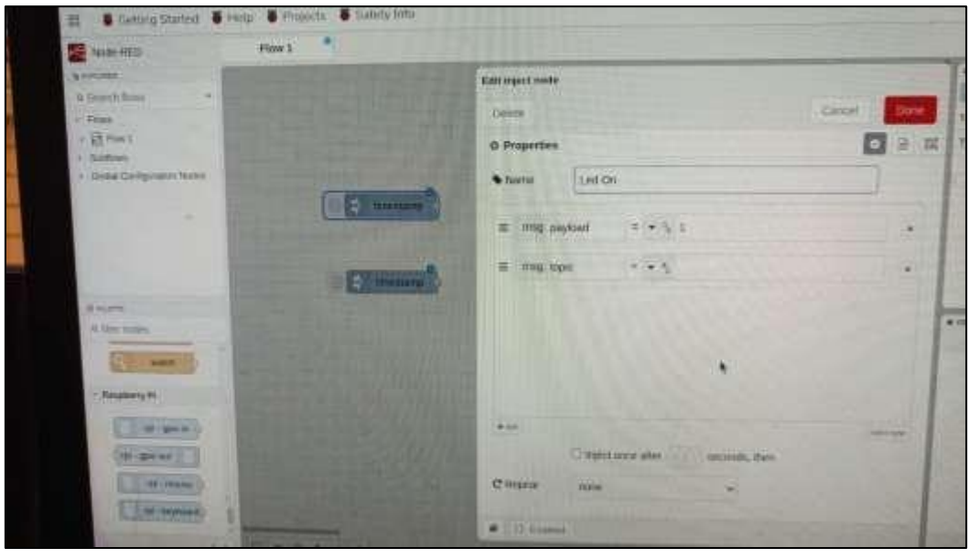


Fig17.3: Take another inject for led off

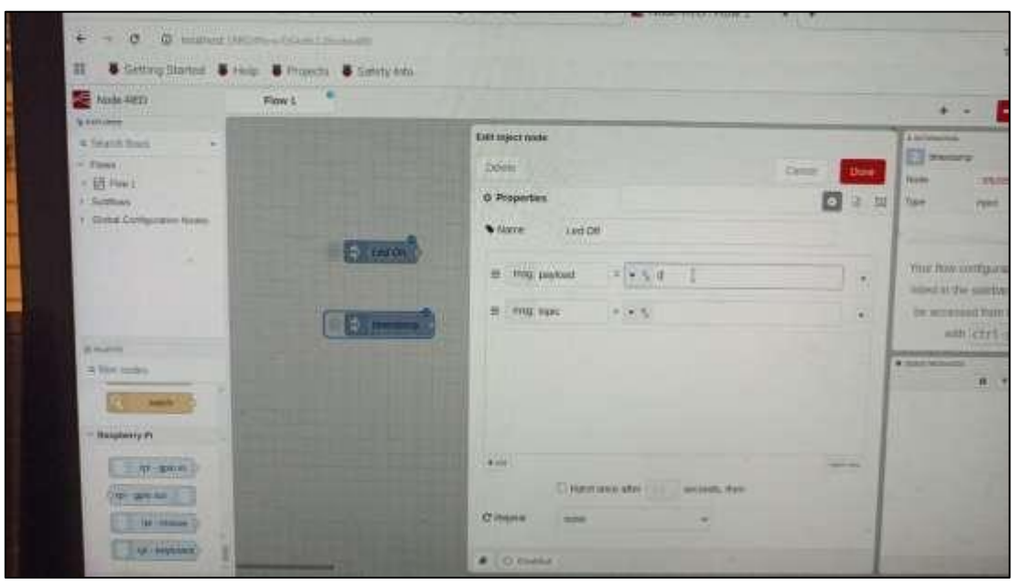


Fig17.4 connection

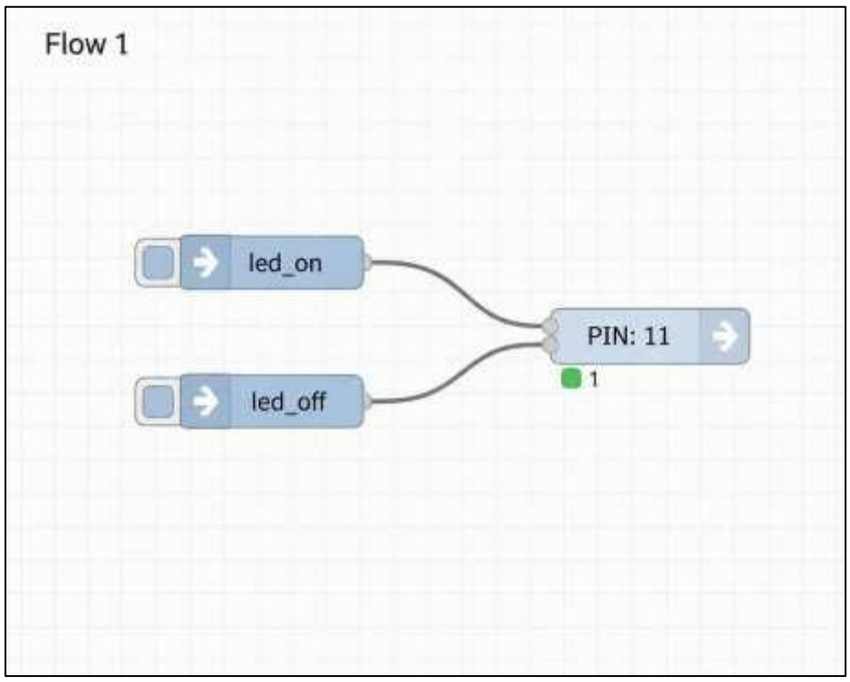


Fig 17.5: led on

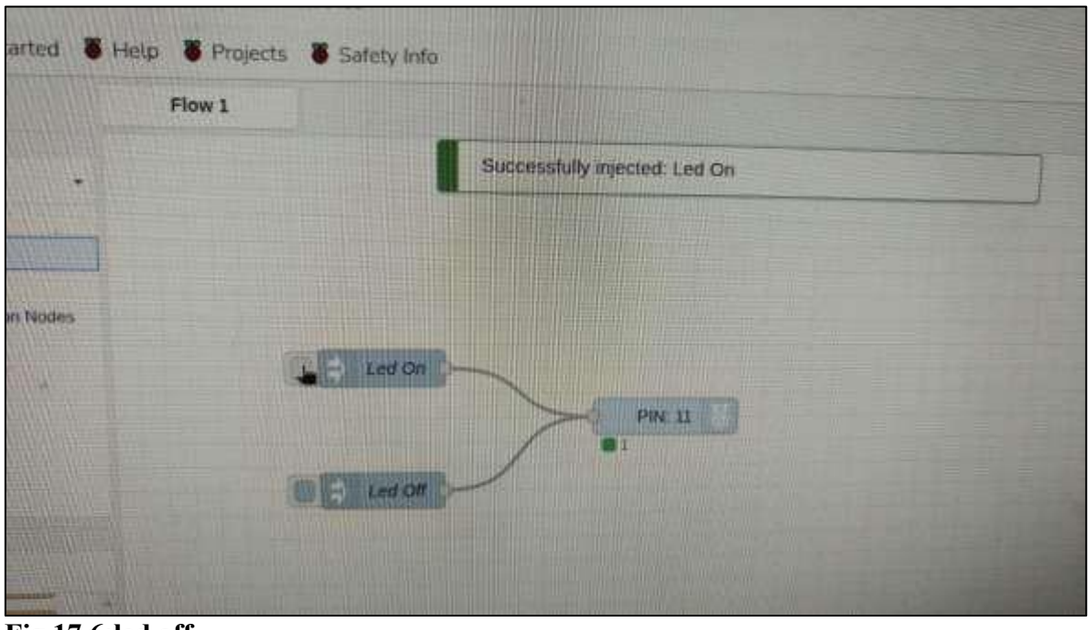
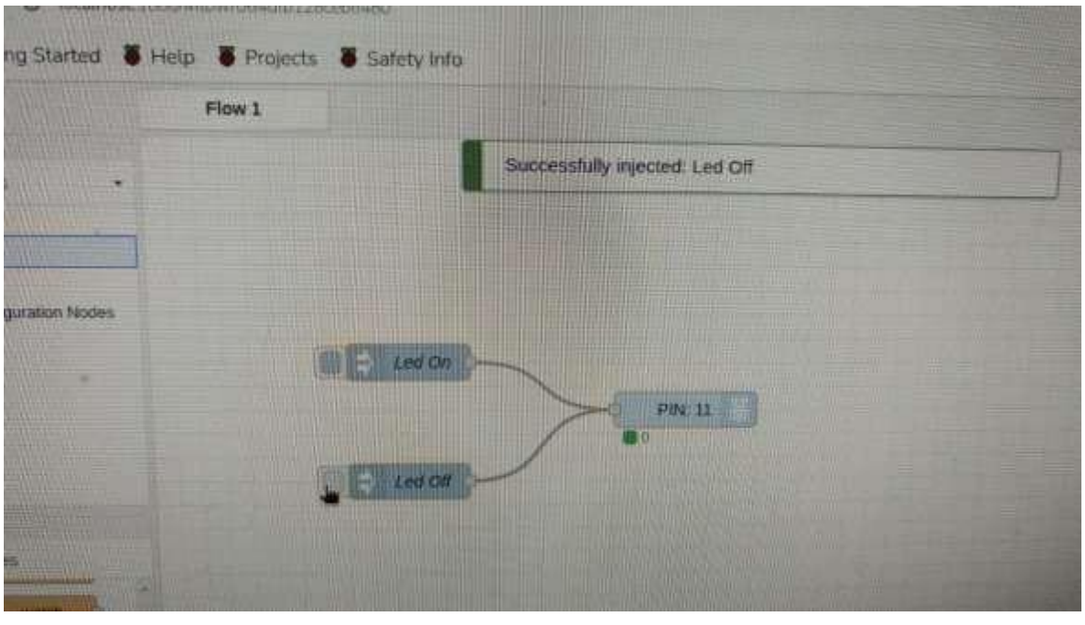


Fig 17.6:led off



Practical No:15

Create two factor authentication system by interfacing fingerprint sensor, RFID and relay with Raspberry Pi.

rc522.py

```
import spidev
import RPi.GPIO as GPIO
import time


class MFRC522:
    CommandReg = 0x01
    CommIEnReg = 0x02
    DivIEnReg = 0x03
    CommIrqReg = 0x04
    DivIrqReg = 0x05
    ErrorReg = 0x06
    Status1Reg = 0x07
    Status2Reg = 0x08
    FIFODataReg = 0x09
    FIFOLevelReg = 0x0A
    WaterLevelReg = 0x0B
    ControlReg = 0x0C
    BitFramingReg = 0x0D
    CollReg = 0x0E


    ModeReg = 0x11
    TxModeReg = 0x12
    RxModeReg = 0x13
    TxControlReg = 0x14
    TxASKReg = 0x15
    TxSelReg = 0x16
    RxSelReg = 0x17
    RxThresholdReg = 0x18
    DemodReg = 0x19


    MfTxReg = 0x1C
    MfRxReg = 0x1D
    SerialSpeedReg = 0x1F


    CRCResultRegH = 0x21
```

```
CRCResultRegL = 0x22
ModWidthReg = 0x24

RFCfgReg = 0x26
GsNReg = 0x27
CWGsPReg = 0x28
ModGsPReg = 0x29

TModeReg = 0x2A
TPrescalerReg = 0x2B
TReloadRegH = 0x2C
TReloadRegL = 0x2D

VersionReg = 0x37

PCD_IDLE = 0x00
PCD_AUTHENT = 0x0E
PCD_TRANSCEIVE = 0x0C
PCD_RESETPHASE = 0x0F
PCD_CALCCRC = 0x03

PICC_REQIDL = 0x26
PICC_ANTICOLL = 0x93
PICC_HALT = 0x50
PICC_AUTHENT1A = 0x60

MI_OK = 0
MI_NOTAGERR = 1
MI_ERR = 2

def __init__(self, spi_bus=0, spi_device=0, rst_pin=25):
    self.rst_pin = rst_pin

    GPIO.setwarnings(False)
    GPIO.setmode(GPIO.BCM)
    GPIO.setup(self.rst_pin, GPIO.OUT)

    self.spi = spidev.SpiDev()
    self.spi.open(spi_bus, spi_device)
```

```
        self.spi.max_speed_hz = 1000000
        self.spi.mode = 0

        self.MFRC522_Init()

    def _write_reg(self, addr, val):
        self.spi.xfer2([((addr << 1) & 0x7E), val])

    def _read_reg(self, addr):
        data = self.spi.xfer2([((addr << 1) & 0x7E) | 0x80, 0])
        return data[1]

    def _set_bitmask(self, reg, mask):
        self._write_reg(reg, self._read_reg(reg) | mask)

    def _clear_bitmask(self, reg, mask):
        self._write_reg(reg, self._read_reg(reg) & (~mask))

    def MFRC522_Reset(self):
        self._write_reg(self.CommandReg, self.PCD_RESETPHASE)

    def AntennaOn(self):
        temp = self._read_reg(self.TxControlReg)

        if ~(temp & 0x03):
            self._set_bitmask(self.TxControlReg, 0x03)

    def AntennaOff(self):
        self._clear_bitmask(self.TxControlReg, 0x03)

    def MFRC522_Init(self):
        GPIO.output(self.rst_pin, GPIO.HIGH)
        time.sleep(0.05)

        self.MFRC522_Reset()

        self._write_reg(self.TModeReg, 0x8D)
        self._write_reg(self.TPrescalerReg, 0x3E)
        self._write_reg(self.TReloadRegL, 30)
```

```
        self._write_reg(self.TReloadRegH, 0)

        self._write_reg(self.TxASKReg, 0x40)
        self._write_reg(self.ModeReg, 0x3D)

        self.AntennaOn()

    def calculate_crc(self, data):
        self._clear_bitmask(self.DivIrqReg, 0x04)
        self._set_bitmask(self.FIFOLevelReg, 0x80)

        for byte in data:
            self._write_reg(self.FIFODataReg, byte)

        self._write_reg(self.CommandReg, self.PCD_CALCCRC)

        for _ in range(255):
            n = self._read_reg(self.DivIrqReg)

            if n & 0x04:
                break

            time.sleep(0.001)

        return [
            self._read_reg(self.CRCResultRegL),
            self._read_reg(self.CRCResultRegH)
        ]

    def ToCard(self, command, send_data):
        back_data = []
        back_len = 0
        status = self.MI_ERR
        irq_en = 0x00
        wait_irq = 0x00

        if command == self.PCD_AUTHENT:
            irq_en = 0x12
            wait_irq = 0x10
```

```
elif command == self.PCD_TRANSCEIVE:
    irq_en = 0x77
    wait_irq = 0x30

self._write_reg(self.CommIEnReg, irq_en | 0x80)
self._clear_bitmask(self.CommIrqReg, 0x80)
self._set_bitmask(self.FIFOLevelReg, 0x80)
self._write_reg(self.CommandReg, self.PCD_IDLE)

for c in send_data:
    self._write_reg(self.FIFODataReg, c)

self._write_reg(self.CommandReg, command)

if command == self.PCD_TRANSCEIVE:
    self._set_bitmask(self.BitFramingReg, 0x80)

i = 2000

while True:
    n = self._read_reg(self.CommIrqReg)
    i -= 1

    if not (i != 0 and not (n & 0x01) and not (n & wait_irq)):
        break

    time.sleep(0.001)

self._clear_bitmask(self.BitFramingReg, 0x80)

if i != 0:
    error = self._read_reg(self.ErrorReg)

    if not (error & 0x1B):
        status = self.MI_OK

        if n & irq_en & 0x01:
            status = self.MI_NOTAGERR
```

```
            if command == self.PCD_TRANSCEIVE:
                n = self._read_reg(self.FIFOLevelReg)

                last_bits = self._read_reg(self.ControlReg) & 0x07

                if last_bits != 0:
                    back_len = (n - 1) * 8 + last_bits
                else:
                    back_len = n * 8

                if n == 0:
                    n = 1

                if n > 16:
                    n = 16

                for _ in range(n):
                    back_data.append(
                        self._read_reg(self.FIFODataReg)
                    )

        return status, back_data, back_len

    def Request(self, req_mode):
        self._write_reg(self.BitFramingReg, 0x07)

        status, back_data, back_bits = self.ToCard(
            self.PCD_TRANSCEIVE,
            [req_mode]
        )

        if status != self.MI_OK or back_bits != 0x10:
            status = self.MI_ERR

        return status, back_bits

    def Anticoll(self):
        self._write_reg(self.BitFramingReg, 0x00)
```

```
    status, back_data, back_bits = self.ToCard(
        self.PCD_TRANSCEIVE,
        [self.PICC_ANTICOLL, 0x20]
    )

    if status == self.MI_OK and len(back_data) == 5:
        checksum = 0

        for i in range(4):
            checksum ^= back_data[i]

        if checksum != back_data[4]:
            status = self.MI_ERR

    return status, back_data

def SelectTag(self, uid):
    buffer = [
        self.PICC_ANTICOLL,
        0x70
    ] + uid

    crc = self.calculate_crc(buffer)
    buffer += crc

    status, back_data, back_bits = self.ToCard(
        self.PCD_TRANSCEIVE,
        buffer
    )

    if status == self.MI_OK and back_bits == 0x18:
        return back_data[0]

    return 0

def HaltA(self):
    buffer = [
        self.PICC_HALT,
```

```
            0x00
        ]

        buffer += self.calculate_crc(buffer)

        self.ToCard(
            self.PCD_TRANSCEIVE,
            buffer
        )

    def read_uid(self):
        status, _ = self.Request(self.PICC_REQIDL)

        if status != self.MI_OK:
            return None

        status, uid = self.Anticoll()

        if status != self.MI_OK:
            return None

        return uid[:4]

    def get_uid_string(self):
        uid = self.read_uid()

        if uid is None:
            return None

        return "-".join(f"{x:02X}" for x in uid)

    def close(self):
        self.AntennaOff()
        self.spi.close()
        GPIO.cleanup(self.rst_pin)
```

rfid_test.py

```
from rc522 import MFRC522
import time

reader = MFRC522()

print("=" * 50)
print("           RFID TEST")
print("=" * 50)

print("MFRC522 initialized.")
print("Place your RFID card near the reader.")
print("Press Ctrl+C to stop.\n")

last_uid = None

try:
    while True:
        uid = reader.get_uid_string()

        if uid:
            if uid != last_uid:
                print("-" * 50)
                print("RFID CARD DETECTED")
                print("-" * 50)
                print("RFID UID:", uid)
                print("RFID test successful.")
                print()

                last_uid = uid

            time.sleep(1)

        else:
            last_uid = None
            time.sleep(0.2)

except KeyboardInterrupt:
    print("\nRFID test stopped.")
```

```
finally:
    reader.close()
```

relay_test.py

```
import RPi.GPIO as GPIO
import time

RELAY_PIN = 17

RELAY_ON = GPIO.LOW
RELAY_OFF = GPIO.HIGH

GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)
GPIO.setup(RELAY_PIN, GPIO.OUT, initial=RELAY_OFF)

print("=" * 50)
print("            RELAY TEST")
print("=" * 50)

try:
    print("Relay OFF")
    GPIO.output(RELAY_PIN, RELAY_OFF)
    time.sleep(2)

    print("Relay ON")
    GPIO.output(RELAY_PIN, RELAY_ON)
    time.sleep(3)

    print("Relay OFF")
    GPIO.output(RELAY_PIN, RELAY_OFF)
    time.sleep(2)

    print("Relay test completed successfully.")

except KeyboardInterrupt:
    print("\nRelay test stopped.")
```

```
finally:
    GPIO.output(RELAY_PIN, RELAY_OFF)
    GPIO.cleanup(RELAY_PIN)
```

fingerprint_test.py

```
import serial
import time
import struct

PORT = "/dev/serial0"
BAUDRATE = 57600

class R307:
    def __init__(self, port=PORT, baudrate=BAUDRATE):
        self.ser = serial.Serial(
            port=port,
            baudrate=baudrate,
            timeout=2
        )

        time.sleep(1)

    def checksum(self, packet):
        return sum(packet) & 0xFFFF

    def send_command(self, command, params=b""):
        address = b"\xFF\xFF\xFF\xFF"
        packet_type = b"\x01"
        length = len(params) + 3

        body = (
            packet_type +
            struct.pack(">H", length) +
            bytes([command]) +
            params
        )

        checksum = self.checksum(body)
```

```
        packet = (
            b"\xEF\x01" +
            address +
            body +
            struct.pack(">H", checksum)
        )

        self.ser.reset_input_buffer()
        self.ser.write(packet)

        return self.read_response()

    def read_response(self):
        header = self.ser.read(9)

        if len(header) < 9:
            return None, b""

        if header[0:2] != b"\xEF\x01":
            return None, b""

        packet_type = header[6]
        length = struct.unpack(">H", header[7:9])[0]

        data = self.ser.read(length)

        if len(data) != length:
            return None, b""

        confirmation = data[0]
        params = data[1:-2]

        return confirmation, params

    def verify_password(self):
        password = b"\x00\x00\x00\x00"

        confirmation, _ = self.send_command(
            0x13,
```

```
        password
    )

    return confirmation == 0x00

def get_image(self):
    confirmation, _ = self.send_command(0x01)

    return confirmation

def image_to_template(self, buffer_id=1):
    confirmation, _ = self.send_command(
        0x02,
        bytes([buffer_id])
    )

    return confirmation

def search(self, buffer_id=1, start_page=0, page_num=1000):
    params = (
        bytes([buffer_id]) +
        struct.pack(">H", start_page) +
        struct.pack(">H", page_num)
    )

    confirmation, data = self.send_command(
        0x04,
        params
    )

    if confirmation != 0x00:
        return None

    if len(data) < 4:
        return None

    finger_id = struct.unpack(">H", data[0:2])[0]
    score = struct.unpack(">H", data[2:4])[0]
```

```
        return finger_id, score

    def close(self):
        self.ser.close()

def wait_for_finger(sensor):
    print("Place your finger on R307...")

    while True:
        result = sensor.get_image()

        if result == 0x00:
            print("Fingerprint detected.")
            return True

        time.sleep(0.2)

sensor = R307()

print("=" * 55)
print("                FINGERPRINT TEST")
print("=" * 55)

try:
    print("Connecting to R307...")

    if sensor.verify_password():
        print("R307 connected.")
        print("Password verification successful.")

    else:
        print("R307 password verification failed.")
        raise SystemExit

    print()
    print("Stored fingerprints: sensor memory available")
    print()
    print("Place a registered finger on the sensor.")
```

```
    wait_for_finger(sensor)

    result = sensor.image_to_template(1)

    if result != 0x00:
        print("Could not convert fingerprint image.")
        raise SystemExit

    result = sensor.search(1, 0, 1000)

    if result:
        finger_id, score = result

        print("Detected fingerprint ID:", finger_id)
        print("Accuracy/Score:", score)
        print("FINGERPRINT MATCHED")
        print("Fingerprint test successful.")

    else:
        print("FINGERPRINT NOT MATCHED")
        print("Fingerprint test failed.")

except KeyboardInterrupt:
    print("\nFingerprint test stopped.")

finally:
    sensor.close()
```

identify_fingerprint.py

```
from fingerprint_test import R307
import time

sensor = R307()

print("=" * 55)
print("            FINGERPRINT IDENTIFICATION")
print("=" * 55)
```

```
try:
    if not sensor.verify_password():
        print("R307 connection failed.")
        raise SystemExit

    print("R307 connected successfully.")
    print("Stored fingerprint database is ready.")
    print()
    print("Place your registered finger on the sensor.")

    while True:

        result = sensor.get_image()

        if result == 0x00:
            print("Fingerprint detected.")

            result = sensor.image_to_template(1)

            if result != 0x00:
                print("Unable to process fingerprint.")
                time.sleep(1)
                continue

            result = sensor.search(1, 0, 1000)

            if result:
                finger_id, score = result

                print("---------------------------------------")
                print("FINGERPRINT MATCHED")
                print("Fingerprint ID:", finger_id)
                print("Accuracy/Score:", score)
                print("---------------------------------------")

            else:
                print("FINGERPRINT NOT FOUND")
```

```
            print("\nReady for next fingerprint...")

            time.sleep(2)

        time.sleep(0.2)

except KeyboardInterrupt:
    print("\nIdentification stopped.")

finally:
    sensor.close()
```

users.json

```
{
    "users": [
        {
            "name": "User 1",
            "rfid_uid": "B9-86-28-03",
            "fingerprint_id": 1
        },
        {
            "name": "User 2",
            "rfid_uid": "12-34-56-78",
            "fingerprint_id": 2
        }
    ]
}
```

access_control.py

```
import json
import time
import RPi.GPIO as GPIO

from rc522 import MFRC522
from fingerprint_test import R307

RELAY_PIN = 17

RELAY_ON = GPIO.LOW
```

```
RELAY_OFF = GPIO.HIGH

USERS_FILE = "users.json"

GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)
GPIO.setup(RELAY_PIN, GPIO.OUT, initial=RELAY_OFF)

def load_users():
    with open(USERS_FILE, "r") as file:
        data = json.load(file)

    return data["users"]

def find_user_by_rfid(users, uid):
    for user in users:
        if user["rfid_uid"].upper() == uid.upper():
            return user

    return None

def authenticate_rfid(reader, users):
    print()
    print("-" * 60)
    print("                  RFID CARD DETECTED")
    print("-" * 60)

    uid = reader.get_uid_string()

    if not uid:
        return None

    print("RFID UID:", uid)

    user = find_user_by_rfid(users, uid)

    if user:
        print("RFID UID MATCHED")
        print("RFID authentication successful.")
```

```
            print("User:", user["name"])

            return user

        print("RFID UID NOT MATCHED")
        print("Access denied.")

        return None

def authenticate_fingerprint(sensor, expected_id):
    print()
    print("=" * 60)
    print("                FINGERPRINT AUTHENTICATION")
    print("=" * 60)

    print("Place your finger on R307...")

    while True:
        result = sensor.get_image()

        if result == 0x00:
            print("Fingerprint detected.")
            break

        time.sleep(0.2)

    result = sensor.image_to_template(1)

    if result != 0x00:
        print("Fingerprint image conversion failed.")
        return False

    result = sensor.search(1, 0, 1000)

    if not result:
        print("FINGERPRINT NOT MATCHED")
        return False

    finger_id, score = result
```

```
    print("Detected fingerprint ID:", finger_id)
    print("Accuracy:", score)

    if finger_id == expected_id:
        print()
        print("FINGERPRINT MATCHED")
        print("Fingerprint ID:", finger_id)

        return True

    print()
    print("FINGERPRINT ID DOES NOT MATCH USER")
    print("Expected ID:", expected_id)
    print("Detected ID:", finger_id)

    return False

def grant_access():
    print()
    print("=" * 60)
    print("                TWO-FACTOR AUTH SUCCESS")
    print("=" * 60)

    print("RFID: MATCHED")
    print("Fingerprint: MATCHED")
    print("ACCESS GRANTED")

    GPIO.output(RELAY_PIN, RELAY_ON)

    print("Relay ON")
    print("Door/lock activated.")

    time.sleep(5)

    GPIO.output(RELAY_PIN, RELAY_OFF)

    print("Relay OFF")
    print("Door/lock secured again.")
```

```
reader = MFRC522()
sensor = R307()

try:
    print("=" * 60)
    print("           RFID + FINGERPRINT + RELAY")
    print("=" * 60)

    print()
    print("Initializing MFRC522...")
    print("MFRC522 initialized.")

    print()
    print("Connecting to R307...")

    if sensor.verify_password():
        print("R307 connected.")
        print("Fingerprint sensor ready.")

    else:
        print("R307 connection failed.")
        raise SystemExit

    users = load_users()

    print()
    print("Users loaded:", len(users))

    print()
    print("=" * 60)
    print("                    SYSTEM READY")
    print("=" * 60)

    print()
    print("Step 1: Scan RFID card.")
    print("Step 2: Scan fingerprint.")
    print()
    print("Press Ctrl+C to stop.")
```

```
last_uid = None

while True:
    uid = reader.get_uid_string()

    if uid:

        if uid != last_uid:

            last_uid = uid

            user = authenticate_rfid(
                reader,
                users
            )

            if user:

                expected_fingerprint = int(
                    user["fingerprint_id"]
                )

                fingerprint_ok = authenticate_fingerprint(
                    sensor,
                    expected_fingerprint
                )

                if fingerprint_ok:
                    grant_access()

                else:
                    print()
                    print("=" * 60)
                    print("            TWO-FACTOR AUTH FAILED")
                    print("=" * 60)

                    print("RFID: MATCHED")
                    print("Fingerprint: NOT MATCHED")
```

```
                print("ACCESS DENIED")

            print()
            print("Ready for next authentication...")

            time.sleep(2)

        else:
            last_uid = None

        time.sleep(0.2)

except KeyboardInterrupt:
    print()
    print("Program stopped.")

finally:
    GPIO.output(RELAY_PIN, RELAY_OFF)

    reader.close()
    sensor.close()

    GPIO.cleanup()

    print("GPIO/SPI/UART closed.")
    print("Program completed.")
```

Practical No:10

Design IoT based home automation system using Raspberry Pi

User Database

```
{
   "users": [
      {
         "name": "User 1",
         "rfid_uid": "B9-86-28-03",
         "fingerprint_id": 1
      },
      {
         "name": "User 2",
         "rfid_uid": "12-34-56-78",
         "fingerprint_id": 2
      }
   ]
}
```

pir_test.py

```
import RPi.GPIO as GPIO
import time

PIR_PIN = 27

GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)
GPIO.setup(PIR_PIN, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)

print("=" * 50)
print(" PIR MOTION SENSOR TEST")
print("=" * 50)
print("Waiting for PIR to settle (10 seconds)...")
time.sleep(10)
print("PIR ready.")
print("Press Ctrl+C to stop.")
print()

try:
    last_state = 0
```

```
    while True:
        state = GPIO.input(PIR_PIN)

        if state == 1 and last_state == 0:
            print("MOTION DETECTED")

        elif state == 0 and last_state == 1:
            print("Motion ended.")

        last_state = state
        time.sleep(0.2)

except KeyboardInterrupt:
    print()
    print("Program stopped.")

finally:
    GPIO.cleanup()
    print("GPIO closed.")
    print("Program complete.")
```

relay_test.py

```
import RPi.GPIO as GPIO
import time

DOOR_RELAY_PIN = 17
LIGHT_RELAY_PIN = 22

RELAY_ON = GPIO.LOW
RELAY_OFF = GPIO.HIGH

GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)

GPIO.setup(
    DOOR_RELAY_PIN,
    GPIO.OUT,
    initial=RELAY_OFF
)
```

```
GPIO.setup(
    LIGHT_RELAY_PIN,
    GPIO.OUT,
    initial=RELAY_OFF
)

print("=" * 50)
print(" 2-CHANNEL RELAY TEST")
print("=" * 50)

try:
    print("Door relay ON")
    GPIO.output(DOOR_RELAY_PIN, RELAY_ON)
    time.sleep(2)

    GPIO.output(DOOR_RELAY_PIN, RELAY_OFF)
    print("Door relay OFF")

    print("Light relay ON")
    GPIO.output(LIGHT_RELAY_PIN, RELAY_ON)
    time.sleep(2)

    GPIO.output(LIGHT_RELAY_PIN, RELAY_OFF)
    print("Light relay OFF")

    print("Relay test completed successfully.")

except KeyboardInterrupt:
    print("Relay test stopped.")

finally:
    GPIO.output(DOOR_RELAY_PIN, RELAY_OFF)
    GPIO.output(LIGHT_RELAY_PIN, RELAY_OFF)
    GPIO.cleanup()
```

home_automation.py

```
import json
import time
import RPi.GPIO as GPIO

from rc522 import MFRC522
from fingerprint_test import R307
from lcd16x2 import lcd_init, lcd_message, lcd_cleanup

DOOR_RELAY_PIN = 17
LIGHT_RELAY_PIN = 22
PIR_PIN = 27

RELAY_ON = GPIO.LOW
RELAY_OFF = GPIO.HIGH

DOOR_OPEN_TIME = 5
LIGHT_HOLD_TIME = 10
FINGER_TIMEOUT = 10

USERS_FILE = "users.json"

GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)

GPIO.setup(
    DOOR_RELAY_PIN,
    GPIO.OUT,
    initial=RELAY_OFF
)

GPIO.setup(
    LIGHT_RELAY_PIN,
    GPIO.OUT,
    initial=RELAY_OFF
)

GPIO.setup(
    PIR_PIN,
```

```
    GPIO.IN,
    pull_up_down=GPIO.PUD_DOWN
)

light_on = False
last_motion = 0.0

def update_light():
    global light_on, last_motion

    now = time.time()

    if GPIO.input(PIR_PIN):
        last_motion = now

        if not light_on:
            light_on = True
            GPIO.output(LIGHT_RELAY_PIN, RELAY_ON)

            print()
            print("-" * 60)
            print(" MOTION DETECTED")
            print("-" * 60)
            print("Light relay ON")

            lcd_message(
                "MOTION DETECTED",
                "LIGHT ON"
            )

    elif light_on and (now - last_motion) > LIGHT_HOLD_TIME:
        light_on = False

        GPIO.output(
            LIGHT_RELAY_PIN,
            RELAY_OFF
        )

        print(
```

```
            "No motion for",
            LIGHT_HOLD_TIME,
            "seconds."
        )

        print("Light relay OFF")

        lcd_message(
            "NO MOTION",
            "LIGHT OFF"
        )

        time.sleep(1)

        lcd_message(
            "SCAN RFID",
            "CARD..."
        )

def wait_with_pir(seconds):
    end = time.time() + seconds

    while time.time() < end:
        update_light()
        time.sleep(0.2)

def load_users():
    with open(USERS_FILE, "r") as file:
        return json.load(file)["users"]

def find_user_by_rfid(users, uid):
    for user in users:
        if user["rfid_uid"].upper() == uid.upper():
            return user

    return None

def authenticate_rfid(reader, users):
    print()
```

```
print("-" * 60)
print(" RFID CARD DETECTED")
print("-" * 60)
print()

uid = reader.get_uid_string()

if not uid:
    return None

print("RFID UID:", uid)

user = find_user_by_rfid(
    users,
    uid
)

if user:
    print("RFID UID MATCHED")
    print("RFID authentication successful.")
    print("User:", user["name"])

    lcd_message(
        "RFID UID",
        "MATCHED"
    )

    wait_with_pir(1.5)

    return user

print("RFID UID NOT MATCHED")
print("Access denied.")

lcd_message(
    "RFID UID",
    "UNMATCHED"
)
```

```
    wait_with_pir(2)

    return None

def authenticate_fingerprint(
    sensor,
    expected_id
):
    print()
    print("=" * 60)
    print(" FINGERPRINT AUTHENTICATION")
    print("=" * 60)
    print()

    print("Place your finger on R307...")

    lcd_message(
        "PLACE FINGER",
        "ON R307..."
    )

    start = time.time()

    while True:
        if sensor.get_image() == 0x00:
            print("Fingerprint detected.")
            break

        if time.time() - start > FINGER_TIMEOUT:
            print("Fingerprint timeout.")
            return False

        update_light()
        time.sleep(0.2)

    if sensor.image_to_template(1) != 0x00:
        print("Fingerprint image conversion failed.")
        return False
```

```
result = sensor.search(
    1,
    0,
    1000
)

if not result:
    print("FINGERPRINT NOT MATCHED")
    return False

finger_id, score = result

print(
    "Detected fingerprint ID:",
    finger_id
)

print(
    "Accuracy:",
    score
)

if finger_id == expected_id:
    print()
    print("FINGERPRINT MATCHED")
    print(
        "Fingerprint ID:",
        finger_id
    )

    return True

print()
print(
    "FINGERPRINT ID DOES NOT MATCH USER"
)

return False
```

```
def grant_access():
    print()
    print("=" * 60)
    print(" TWO-FACTOR AUTH SUCCESS")
    print("=" * 60)
    print()

    print("RFID: MATCHED")
    print("Fingerprint: MATCHED")
    print("ACCESS GRANTED")

    lcd_message(
        "ACCESS GRANTED",
        "DOOR OPEN"
    )

    GPIO.output(
        DOOR_RELAY_PIN,
        RELAY_ON
    )

    print("Door relay ON")

    wait_with_pir(
        DOOR_OPEN_TIME
    )

    GPIO.output(
        DOOR_RELAY_PIN,
        RELAY_OFF
    )

    print("Door relay OFF")
    print("Door secured again.")

    lcd_message(
        "DOOR LOCKED",
        "SECURED"
    )
```

```
    wait_with_pir(1.5)

def deny_access():
    print()
    print("=" * 60)
    print(" TWO-FACTOR AUTH FAILED")
    print("=" * 60)
    print()

    print("RFID: MATCHED")
    print("Fingerprint: NOT MATCHED")
    print("ACCESS DENIED")

    lcd_message(
        "ACCESS DENIED",
        "FINGER WRONG"
    )

    wait_with_pir(2)

print()
print("=" * 60)
print(" IoT HOME AUTOMATION SYSTEM")
print(" RFID + FINGERPRINT + PIR + RELAY + I2C LCD")
print("=" * 60)
print()

print("Initializing I2C LCD...")

lcd_init()

print("LCD initialized.")

print("Initializing MFRC522...")

reader = MFRC522()

print("MFRC522 initialized.")
```

```
print("Connecting to R307...")

sensor = R307()

if not sensor.verify_password():
    print("R307 connection failed.")
    raise SystemExit

print("R307 connected.")

print(
    "Initializing PIR sensor "
    "(settling 10 seconds)..."
)

lcd_message(
    "PIR WARMING UP",
    "PLEASE WAIT"
)

time.sleep(10)

print("PIR sensor ready.")

users = load_users()

print(
    "Users loaded:",
    len(users)
)

print()
print("=" * 60)
print(" SYSTEM READY")
print("=" * 60)
print()

print(
```

```
    "Door    : Scan RFID card, then fingerprint."
)

print(
    "Light   : Automatic with PIR motion sensor."
)

print()
print("Press Ctrl+C to stop.")

lcd_message(
    "HOME AUTOMATION",
    "SYSTEM READY"
)

time.sleep(2)

lcd_message(
    "SCAN RFID",
    "CARD..."
)

try:
    last_uid = None

    while True:
        update_light()

        uid = reader.get_uid_string()

        if uid:

            if uid != last_uid:

                last_uid = uid

                user = authenticate_rfid(
                    reader,
                    users
```

```
            )

            if user:

                expected = int(
                    user["fingerprint_id"]
                )

                if authenticate_fingerprint(
                    sensor,
                    expected
                ):
                    grant_access()

                else:
                    deny_access()

            print()
            print(
                "Ready for next authentication."
            )

            lcd_message(
                "SCAN RFID",
                "CARD..."
            )

            time.sleep(1)

        else:
            last_uid = None

        time.sleep(0.2)

except KeyboardInterrupt:
    print()
    print("Program stopped.")

finally:
```

```
    GPIO.output(
        DOOR_RELAY_PIN,
        RELAY_OFF
    )

    GPIO.output(
        LIGHT_RELAY_PIN,
        RELAY_OFF
    )

    lcd_cleanup()
    reader.close()
    sensor.close()
    GPIO.cleanup()

    print(
        "GPIO/SPI/I2C/UART closed."
    )

    print("Program complete.")
```

Output

```
Iot2026@raspberrypi:~/two_factor_auth $ source home-env/bin/activate

(home-env) iot2026@raspberrypi:~/two_factor_auth $ python3 home_automation.py

============================================================
 IoT HOME AUTOMATION SYSTEM
 RFID + FINGERPRINT + PIR + RELAY + I2C LCD
============================================================

Initializing I2C LCD...
LCD initialized.
Initializing MFRC522...
MFRC522 initialized.
Connecting to R307...
R307 connected.
Initializing PIR sensor (settling 10 seconds)...
PIR sensor ready.
```

```
Users loaded: 2

============================================================
 SYSTEM READY
============================================================

Door    : Scan RFID card, then fingerprint.
Light   : Automatic with PIR motion sensor.

Press Ctrl+C to stop.

------------------------------------------------------------
 MOTION DETECTED
------------------------------------------------------------
Light relay ON

------------------------------------------------------------
 RFID CARD DETECTED
------------------------------------------------------------

RFID UID: B9-86-28-03
RFID UID MATCHED
RFID authentication successful.
User: User 1

============================================================
 FINGERPRINT AUTHENTICATION
============================================================

Place your finger on R307...
Fingerprint detected.
Detected fingerprint ID: 1
Accuracy: 85

FINGERPRINT MATCHED
Fingerprint ID: 1

============================================================
 TWO-FACTOR AUTH SUCCESS
```

```
============================================================

RFID: MATCHED
Fingerprint: MATCHED
ACCESS GRANTED
Door relay ON
Door relay OFF
Door secured again.

Ready for next authentication.

No motion for 10 seconds.
Light relay OFF

------------------------------------------------------------
 RFID CARD DETECTED
------------------------------------------------------------

RFID UID: A1-B2-C3-D4
RFID UID NOT MATCHED
Access denied.

Ready for next authentication.

------------------------------------------------------------
 RFID CARD DETECTED
------------------------------------------------------------

RFID UID: B9-86-28-03
RFID UID MATCHED
RFID authentication successful.
User: User 1

============================================================
 FINGERPRINT AUTHENTICATION
============================================================

Place your finger on R307...
Fingerprint detected.
```

```
Detected fingerprint ID: 2
Accuracy: 71

FINGERPRINT ID DOES NOT MATCH USER

============================================================
 TWO-FACTOR AUTH FAILED
============================================================

RFID: MATCHED
Fingerprint: NOT MATCHED
ACCESS DENIED

Ready for next authentication.

------------------------------------------------------------
 MOTION DETECTED
------------------------------------------------------------
Light relay ON

No motion for 10 seconds.
Light relay OFF

^C

Program stopped.
GPIO/SPI/I2C/UART closed.
Program complete.
```

Practical No:19

Design and implement an embedded access-control prototype that grants entry only when either a whitelisted RFID tag is detected or a legitimate fingerprint is verified, provides real-time visual feedback on an LCD, indicates status via an LED, and pulses a relay (active-LOW) to unlock for a fixed interval. The system must log all events (time, method, result) for auditability.

App,py

```
import time
import threading
from datetime import datetime
import serial
import mysql.connector
import adafruit_fingerprint
from flask import Flask, jsonify, render_template_string
from mfrc522 import SimpleMFRC522
from RPLCD.i2c import CharLCD
from gpiozero import OutputDevice, LED

app = Flask(__name__)

DB_CONFIG = {
    "host": "localhost",
    "user": "access_app",
    "password": "AccessPi2026!",
    "database": "access_control_db"
}

FINGERPRINT_PORT = "/dev/ttyAMA0"
FINGERPRINT_BAUD = 57600
LCD_ADDRESS = 0x27
LCD_PORT = 1
RELAY_GPIO = 17
LED_GPIO = 27
UNLOCK_SECONDS = 5

relay = OutputDevice(
    RELAY_GPIO,
    active_high=False,
    initial_value=False
)
```

```
led = LED(LED_GPIO)

lcd = None
finger = None
process_lock = threading.Lock()

status = {
    "database": "Checking...",
    "rfid": "Waiting",
    "fingerprint": "Waiting",
    "access": "WAITING",
    "user": "-",
    "method": "-",
    "credential": "-",
    "relay": "LOCKED",
    "led": "OFF",
    "reason": "Ready to scan",
    "message": "Scan an RFID card or place your finger",
    "time": "-"
}



def update_status(**values):
    status.update(values)
    status["time"] = datetime.now().strftime("%Y-%m-%d %H:%M:%S")



def show_lcd(line1, line2=""):
    global lcd

    if lcd is None:
        return

    try:
        lcd.clear()
        lcd.write_string(str(line1)[:16])
        lcd.cursor_pos = (1, 0)
        lcd.write_string(str(line2)[:16])
```

```
    except Exception as exc:
        print("LCD error:", exc)


def lock_door():
    try:
        relay.off()
        led.off()

        update_status(
            relay="LOCKED",
            led="OFF"
        )

        print("Door locked")

    except Exception as exc:
        print("Lock error:", exc)


def unlock_door():
    relay.on()
    led.on()

    update_status(
        relay="UNLOCKED",
        led="ON"
    )

    print("Door unlocked")


def get_db_connection():
    return mysql.connector.connect(**DB_CONFIG)


def check_database():
    connection = None
```

```
    try:
        connection = get_db_connection()
        cursor = connection.cursor()

        cursor.execute("SELECT 1")
        cursor.fetchone()

        cursor.close()

        update_status(database="Connected")

        return True

    except Exception as exc:
        print("Database connection error:", exc)

        update_status(database="Disconnected")

        return False

    finally:
        if connection and connection.is_connected():
            connection.close()



def authorize_and_log(method, credential):
    connection = None
    cursor = None

    try:
        connection = get_db_connection()

        cursor = connection.cursor(dictionary=True)

        cursor.execute(
            """
            SELECT u.user_id, u.name
            FROM credentials AS c
            JOIN users AS u ON u.user_id = c.user_id
```

```
    WHERE c.method = %s
      AND c.credential_value = %s
      AND c.active = TRUE
      AND u.active = TRUE
    LIMIT 1
    """,
    (method, str(credential))
)

user = cursor.fetchone()

if user:
    result = "GRANTED"
    user_id = user["user_id"]
    user_name = user["name"]
    reason = "Authorized credential"
else:
    result = "DENIED"
    user_id = None
    user_name = "-"
    reason = "Credential not registered or inactive"

cursor.execute(
    """
    INSERT INTO access_logs
        (user_id, method, credential_value, result, reason)
    VALUES (%s, %s, %s, %s, %s)
    """,
    (
        user_id,
        method,
        str(credential),
        result,
        reason
    )
)

connection.commit()
```

```
        update_status(
            database="Connected",
            user=user_name,
            method=method,
            credential=str(credential),
            access=result,
            reason=reason,
            message=(
                "Welcome, " + user_name
                if result == "GRANTED"
                else "Access denied"
            )
        )

        return result == "GRANTED", user_name, reason

    except Exception as exc:
        print("Database error:", exc)

        if connection:
            try:
                connection.rollback()
            except Exception:
                pass

        update_status(
            database="Error",
            user="-",
            method=method,
            credential=str(credential),
            access="DENIED",
            reason="Database error",
            message="Database error - access blocked"
        )

        return False, "-", "Database error"

    finally:
        if cursor:
```

```
            cursor.close()

        if connection and connection.is_connected():
            connection.close()



def process_credential(method, credential):
    with process_lock:

        print(f"Checking {method}: {credential}")

        if method == "RFID":
            update_status(rfid="Card detected")
        else:
            update_status(fingerprint="Finger detected")

        show_lcd("Checking access", method)

        allowed, user_name, reason = authorize_and_log(
            method,
            str(credential)
        )

        if allowed:

            show_lcd("ACCESS GRANTED", user_name)

            print("ACCESS GRANTED:", user_name)

            try:
                unlock_door()

                update_status(
                    access="GRANTED",
                    user=user_name,
                    method=method,
                    message="Welcome, " + user_name,
                    reason="Authorized credential"
                )
```

```
            time.sleep(UNLOCK_SECONDS)

        except Exception as exc:

            print("Relay operation error:", exc)

            update_status(
                access="DENIED",
                relay="LOCKED",
                led="OFF",
                message="Hardware error",
                reason="Relay operation failed"
            )

        finally:
            lock_door()

    else:

        show_lcd("ACCESS DENIED", "Try again")

        print("ACCESS DENIED:", reason)

        update_status(
            access="DENIED",
            relay="LOCKED",
            led="OFF",
            user="-",
            message="Access denied",
            reason=reason
        )

        lock_door()

        time.sleep(2)

    if method == "RFID":
        update_status(rfid="Waiting")
```

```
        else:
            update_status(fingerprint="Waiting")

        update_status(
            message="Scan an RFID card or place your finger"
        )


def rfid_worker():

    try:
        reader = SimpleMFRC522()

        update_status(rfid="Ready")

        print("RFID reader ready")

    except Exception as exc:

        print("RFID initialization error:", exc)

        update_status(rfid="Hardware error")

        return

    while True:

        try:
            uid, text = reader.read()

            credential = str(uid)

            print("RFID UID:", credential)

            process_credential(
                "RFID",
                credential
            )
```

```
        except Exception as exc:

            print("RFID reading error:", exc)

            update_status(rfid="Read error")

            time.sleep(1)



def wait_for_finger_removed(sensor):

    while True:

        try:
            result = sensor.get_image()

            if result != adafruit_fingerprint.OK:
                break

            time.sleep(0.3)

        except Exception:
            break



def fingerprint_worker():

    global finger

    uart = None

    try:

        uart = serial.Serial(
            FINGERPRINT_PORT,
            baudrate=FINGERPRINT_BAUD,
            timeout=2
        )
```

```
    finger = adafruit_fingerprint.Adafruit_Fingerprint(uart)

    if not finger.verify_password():

        print(
            "Fingerprint sensor password verification failed"
        )

        update_status(
            fingerprint="Sensor error"
        )

        return

    print("R307 fingerprint sensor ready")

    print(
        "Stored template IDs:",
        finger.read_templates()
    )

    update_status(
        fingerprint="Ready"
    )

except Exception as exc:

    print(
        "Fingerprint initialization error:",
        exc
    )

    update_status(
        fingerprint="Hardware error"
    )

    if uart and uart.is_open:
        uart.close()
```

```
    return

while True:

    try:

        result = finger.get_image()

        if result != adafruit_fingerprint.OK:

            time.sleep(0.2)

            continue

        print("Fingerprint detected")

        update_status(
            fingerprint="Processing"
        )

        show_lcd(
            "Fingerprint",
            "Checking..."
        )

        result = finger.image_2_tz(1)

        if result != adafruit_fingerprint.OK:

            update_status(
                fingerprint="Try again",
                message="Could not read fingerprint"
            )

            wait_for_finger_removed(finger)

            continue

        result = finger.finger_search()
```

```
    if result == adafruit_fingerprint.OK:

        template_id = str(
            finger.finger_id
        )

        print(
            "Fingerprint template ID:",
            template_id
        )

        process_credential(
            "FINGERPRINT",
            template_id
        )

    else:

        process_credential(
            "FINGERPRINT",
            "NO_MATCH"
        )

    wait_for_finger_removed(finger)

except Exception as exc:

    print(
        "Fingerprint reading error:",
        exc
    )

    update_status(
        fingerprint="Read error"
    )

    time.sleep(1)
```

```
HTML_PAGE = """
<!DOCTYPE html>
<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta
    name="viewport"
    content="width=device-width, initial-scale=1"
  >

  <title>Smart Door Lock</title>

  <style>

    body {
      margin: 0;
      padding: 20px;
      font-family: Arial, sans-serif;
      background: #f1f4f8;
      color: #222;
    }

    .card {
      max-width: 420px;
      margin: 30px auto;
      padding: 24px;
      background: white;
      border-radius: 12px;
      box-shadow: 0 3px 14px #d9dfe7;
    }

    h1 {
      text-align: center;
      margin-bottom: 6px;
      font-size: 25px;
```

```
}

.subtitle {
    text-align: center;
    color: #777;
    margin-bottom: 25px;
}

.row {
    display: flex;
    justify-content: space-between;
    gap: 12px;
    padding: 13px 0;
    border-bottom: 1px solid #eee;
}

.value {
    font-weight: bold;
    text-align: right;
    overflow-wrap: anywhere;
}

.door {
    margin-top: 25px;
    padding: 20px;
    text-align: center;
    border-radius: 10px;
    background: #eeeeee;
}

.granted {
    background: #d9f8e2;
    color: #176b35;
}

.denied {
    background: #ffe0e0;
    color: #a40000;
}
```

```
        .locked {
            background: #eeeeee;
            color: #333333;
        }

        .message {
            margin-top: 20px;
            text-align: center;
            color: #666;
            font-size: 14px;
        }

    </style>

</head>

<body>

    <div class="card">

        <h1>Smart Door Lock</h1>

        <div class="subtitle">
            Raspberry Pi 5 Access Control
        </div>

        <div class="row">
            <span>Database</span>
            <span class="value" id="database">
                Checking...
            </span>
        </div>

        <div class="row">
            <span>RFID Card</span>
            <span class="value" id="rfid">
                Waiting
            </span>
```

```
</div>

<div class="row">
  <span>Fingerprint</span>
  <span class="value" id="fingerprint">
    Waiting
  </span>
</div>

<div class="row">
  <span>Recognized User</span>
  <span class="value" id="user">
    -
  </span>
</div>

<div class="row">
  <span>Authentication</span>
  <span class="value" id="method">
    -
  </span>
</div>

<div class="row">
  <span>Indicator LED</span>
  <span class="value" id="led">
    OFF
  </span>
</div>

<div class="door locked" id="door">

  <h2 id="access">
    DOOR LOCKED
  </h2>

  <p id="relay">
    Relay: LOCKED
  </p>
```

```
    </div>

    <div class="message" id="message">
        Scan an RFID card or place your finger
    </div>

</div>

<script>

    async function refreshStatus() {

        try {

            const response =
                await fetch('/api/data');

            if (!response.ok)
                throw new Error('Request failed');

            const s =
                await response.json();

            document.getElementById(
                'database'
            ).textContent =
                s.database || 'Unknown';

            document.getElementById(
                'rfid'
            ).textContent =
                s.rfid || 'Waiting';

            document.getElementById(
                'fingerprint'
            ).textContent =
                s.fingerprint || 'Waiting';
```

```
document.getElementById(
    'user'
).textContent =
    s.user || '-';

document.getElementById(
    'method'
).textContent =
    s.method || '-';

document.getElementById(
    'led'
).textContent =
    s.led || 'OFF';

document.getElementById(
    'relay'
).textContent =
    'Relay: ' +
    (s.relay || 'LOCKED');

document.getElementById(
    'message'
).textContent =
    s.message ||
    'System running';

const door =
    document.getElementById(
        'door'
    );

const access =
    document.getElementById(
        'access'
    );

if (s.access === 'GRANTED') {
```

```
            access.textContent =
                'ACCESS GRANTED';

            door.className =
                'door granted';

        } else if (
            s.access === 'DENIED'
        ) {

            access.textContent =
                'ACCESS DENIED';

            door.className =
                'door denied';

        } else {

            access.textContent =
                s.relay === 'UNLOCKED'
                ? 'DOOR UNLOCKED'
                : 'DOOR LOCKED';

            door.className =
                'door locked';
        }

    } catch (error) {

        document.getElementById(
            'database'
        ).textContent =
            'Connection error';

        document.getElementById(
            'message'
        ).textContent =
            'Check the Raspberry Pi application.';
    }
```

```
        }

        refreshStatus();

        setInterval(
            refreshStatus,
            1000
        );

    </script>

</body>

</html>
"""



@app.route("/")
def home():
    return render_template_string(
        HTML_PAGE
    )



@app.route("/api/data")
def api_data():
    return jsonify(status)



if __name__ == "__main__":

    print(
        "Starting Smart Door Lock System..."
    )

    try:

        lcd = CharLCD(
            i2c_expander="PCF8574",
```

```
        address=LCD_ADDRESS,
        port=LCD_PORT,
        cols=16,
        rows=2
    )

    show_lcd(
        "Smart Door Lock",
        "Starting..."
    )

except Exception as exc:

    print(
        "LCD initialization error:",
        exc
    )

lock_door()

check_database()

update_status(
    message="Scan an RFID card or place your finger",
    access="WAITING",
    user="-",
    method="-"
)

show_lcd(
    "Smart Door Lock",
    "Ready to scan"
)

threading.Thread(
    target=rfid_worker,
    daemon=True
).start()
```

```
threading.Thread(
    target=fingerprint_worker,
    daemon=True
).start()

print(
    "Webpage: http://RASPBERRY_PI_IP:5000"
)

app.run(
    host="0.0.0.0",
    port=5000,
    debug=False,
    use_reloader=False
)
```

Practical No:13

Implement basic Publisher – Subscriber using MQTT. Publish messages using paho mqtt. Subscribe and control LED

Subscriber.py

```
import paho.mqtt.client as mqtt
from gpiozero import LED
E
BROKER = "localhost"
PORT = 1883
TOPIC = "home/led"


led = LED(17)


def on_connect(client, userdata, flags, rc):
    if rc == 0:
        print("Connected to MQTT Broker")
        client.subscribe(TOPIC)
        print("Subscribed to:", TOPIC)
    else:
        print("MQTT connection failed")


def on_message(client, userdata, msg):
    message = msg.payload.decode().strip().upper()


    print("Received:", message)


    if message == "ON":
        led.on()
        print("LED ON")


    elif message == "OFF":
        led.off()
        print("LED OFF")


    else:
        print("Unknown message")


client = mqtt.Client()
```

```
client.on_connect = on_connect
client.on_message = on_message

client.connect(BROKER, PORT, 60)

print("MQTT Subscriber Started")
print("Waiting for messages...")

client.loop_forever()
```

Publisher.py

```
import paho.mqtt.client as mqtt

BROKER = "192.168.1.160"
PORT = 1883
TOPIC = "home/led"

client = mqtt.Client()

client.connect(BROKER, PORT, 60)

print("MQTT Publisher Started")
print("Type ON or OFF")
print("Type EXIT to stop")

while True:
    message = input("Enter message: ").strip().upper()

    if message == "EXIT":
        break

    if message in ["ON", "OFF"]:
        client.publish(TOPIC, message)
        print("Published:", message)

    else:
        print("Please enter ON, OFF or EXIT")

client.disconnect()
```

Output

MQTT Subscriber Started

Waiting for messages...

Connected to MQTT Broker

Subscribed to: home/led

Received: ON

LED ON

Received: OFF

LED OFF